Lecture 6 Bottom-up Parsing

Instructor: Dr. Jianhui Yue

CS 4121 Fall 2026

Michigan Tech. University

Material adapted from Keith Schwarz, and Steve Zdancewic slides.

Bottom-up Parsing (Objectives)

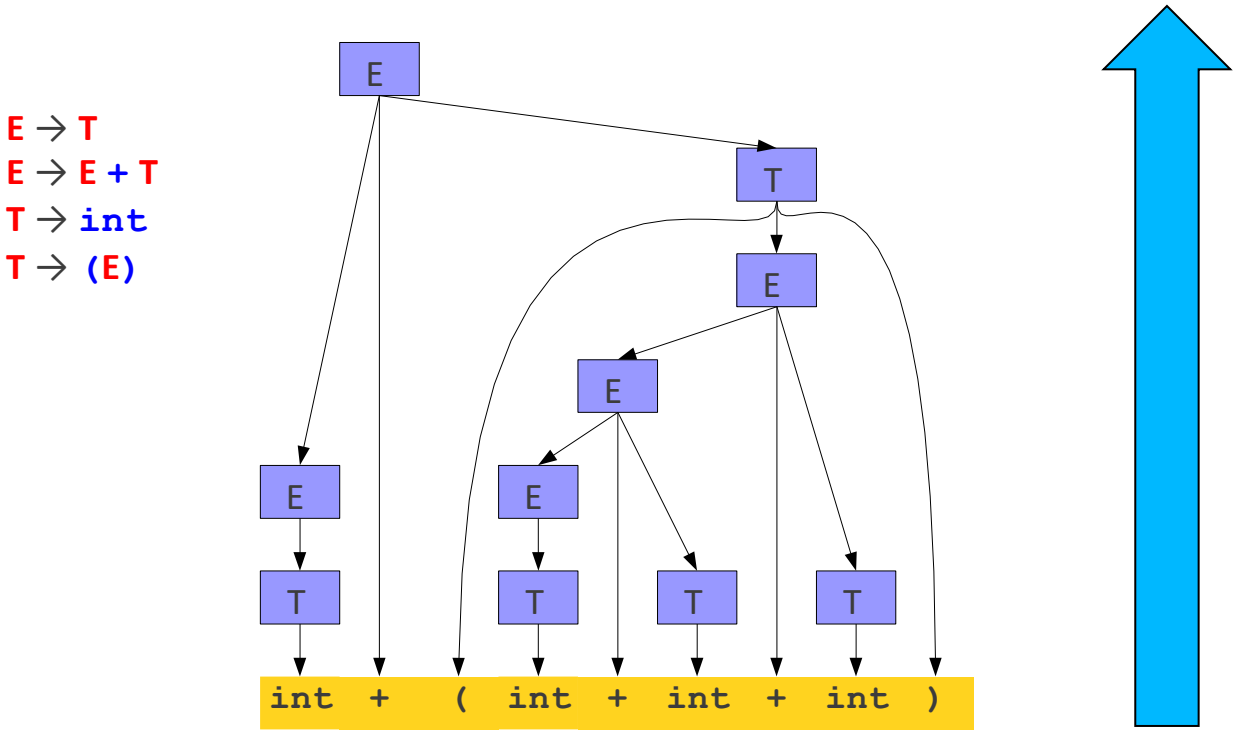
- Given a parse table and an input string, the student will be able to apply shift-reduce parsing to the string using the parse table.
- Given a context-free grammar, the student will be able to determine if it is LR/SLR by constructing its corresponding parse table.

Outline

1. Intro. to bottom-up parsing
2. LR(0) parsing
3. LR(1) and SLR(1) parsing

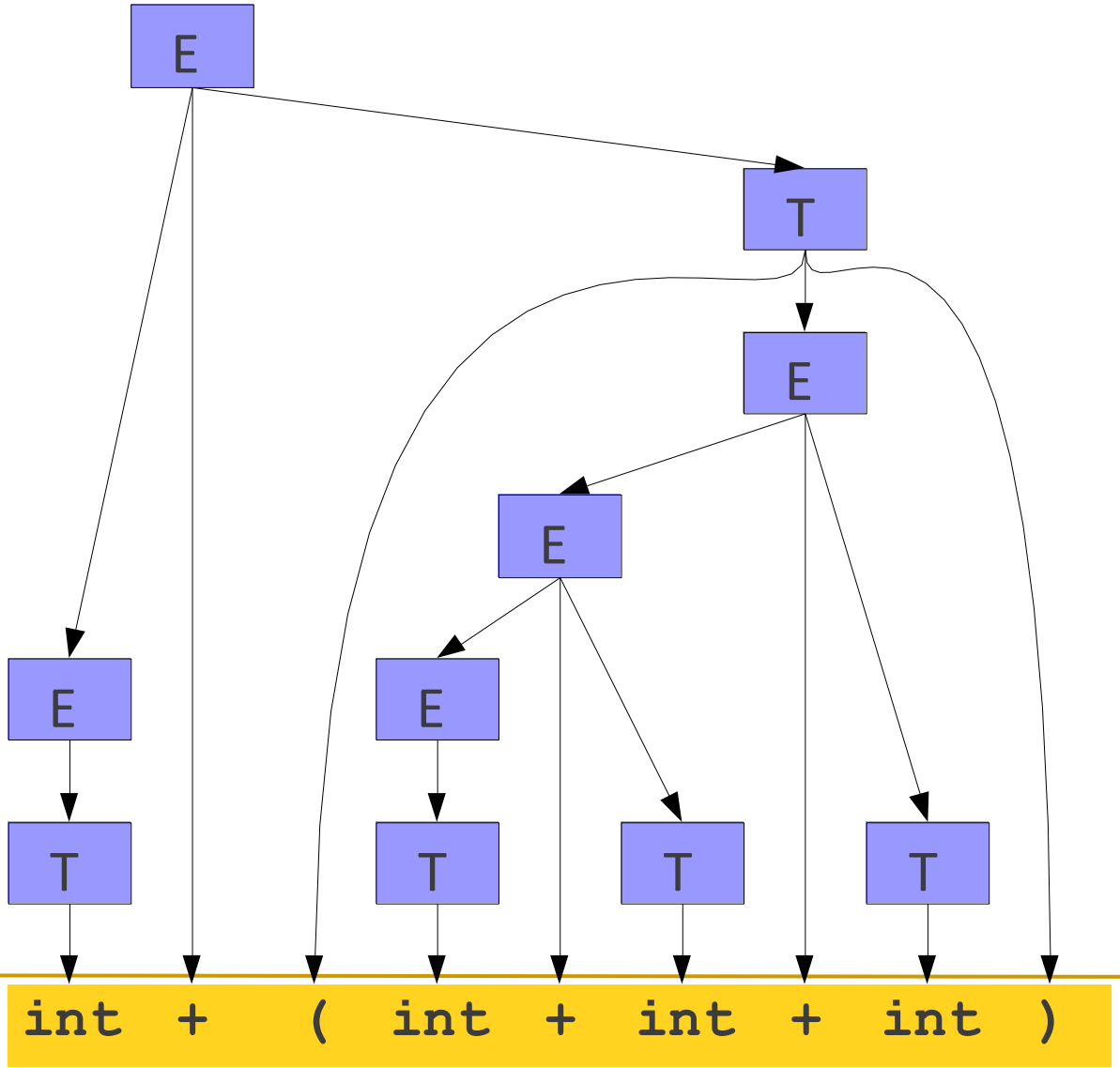
What is Bottom-Up Parsing?

Idea: Apply productions **in reverse** to convert the user's program to the start symbol.



One View of a Bottom-Up Parse

E → T
E → E + T
T → int
T → (E)



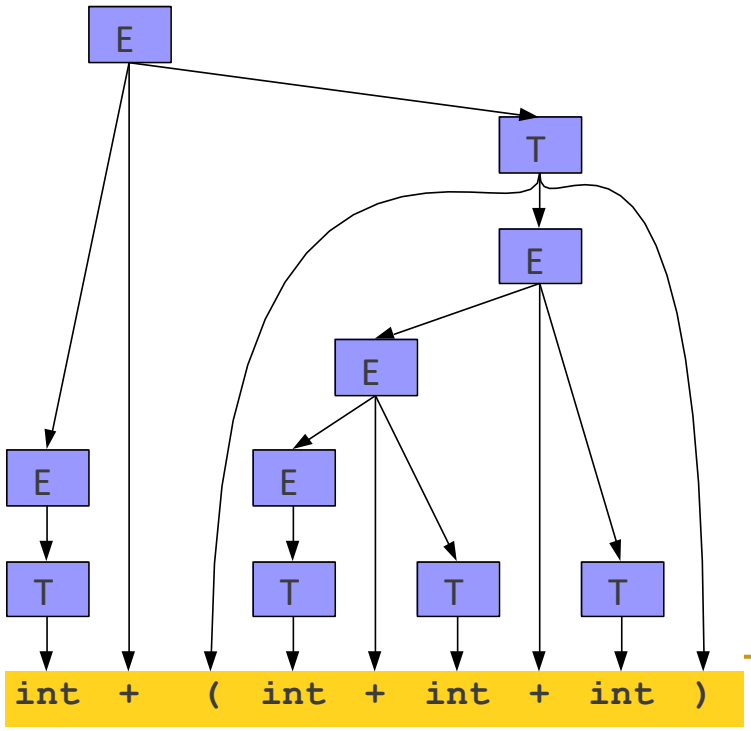
A Second View of a Bottom-Up Parse

$E \rightarrow T$

$E \rightarrow E + T$

$T \rightarrow int$

$T \rightarrow (E)$



int + (int + int + int)

⇒ T + (int + int + int)

⇒ E + (int + int + int)

⇒ E + (T + int + int)

⇒ E + (E + int + int)

⇒ E + (E +T + int)

⇒ E + (E + int)

⇒ E + (E + T)

⇒ E + (E)

⇒ E + T

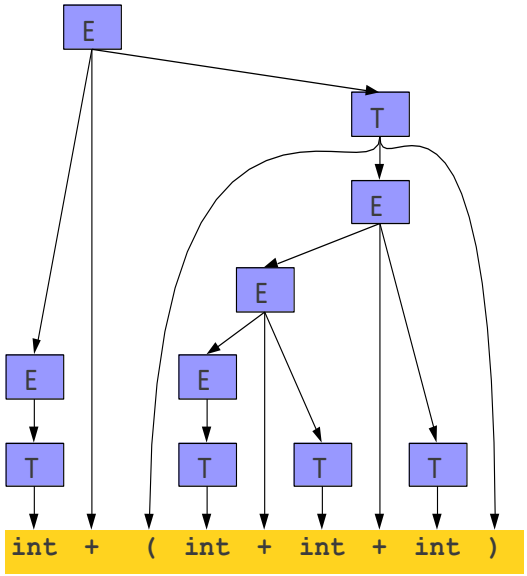
⇒ E

A Second View of a Bottom-Up Parse

A left-to-right, bottom-up parse is a rightmost derivation traced in reverse.

E → T
E → E + T
T → int
T → (E)

Reduction

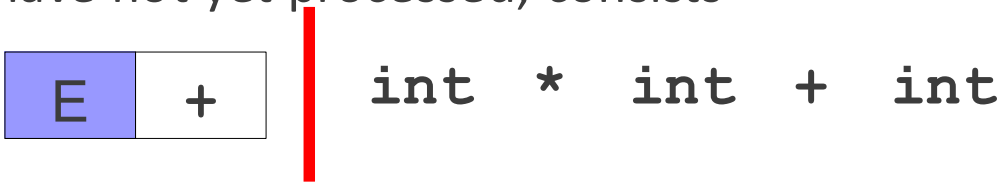


int + (int + int + int)
⇒ **T** + (int + int + int)
⇒ **E** + (int + int + int)
⇒ E + (**T** + int + int)
⇒ E + (**E** + int + int)
⇒ E + (E + **T** + int)
⇒ E + (**E** + int)
⇒ E + (E + **T**)
⇒ E + (**E**)
⇒ E + **T**
⇒ **E**

Derivation

Shift/Reduce Parsing

- The bottom-up parsers we will consider are called **shift/reduce** parsers.
  - Contrast with the LL(1) **predict/match** parser.
- Idea: Split the input into two parts:
  - Left substring is our work area; all **handles** must be here.
  - Right substring is input we have not yet processed; consists purely of terminals.



- At each point, decide whether to:
  - Move a terminal across the split (**shift**)
  - Reduce a handle (**reduce**)

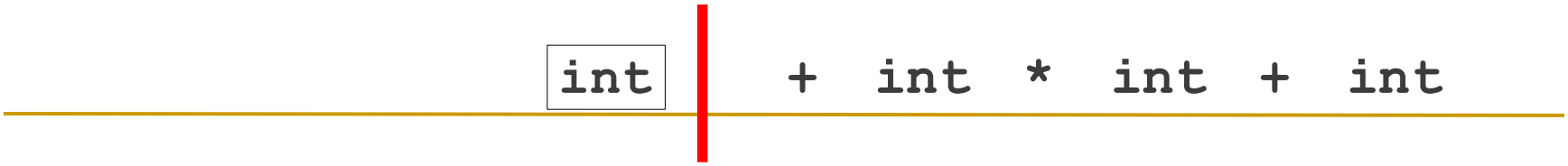
A Sample Shift/Reduce Parse

E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

	int + int * int + int

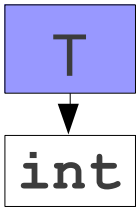
A Sample Shift/Reduce Parse

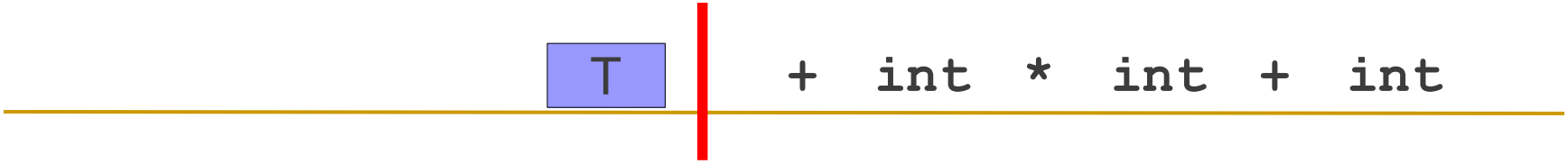
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



A Sample Shift/Reduce Parse

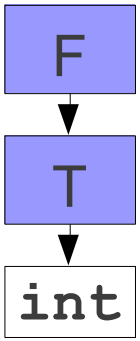
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)





A Sample Shift/Reduce Parse

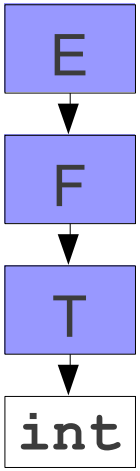
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



F | + int * int + int

A Sample Shift/Reduce Parse

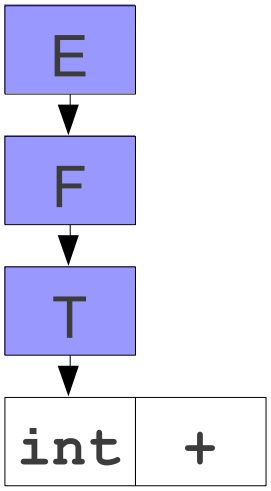
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



E | + int * int + int

A Sample Shift/Reduce Parse

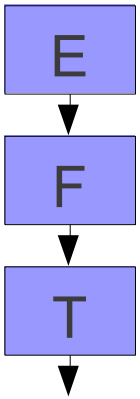
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



int * int + int

A Sample Shift/Reduce Parse

E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

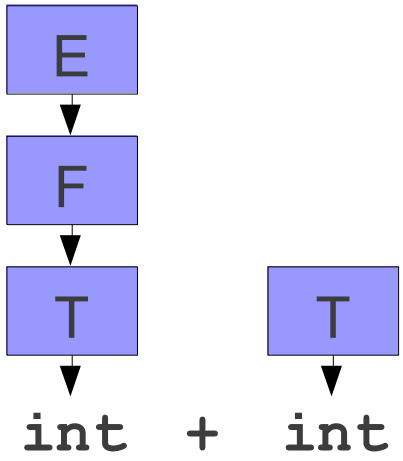


int + int

E + int | * int + int

A Sample Shift/Reduce Parse

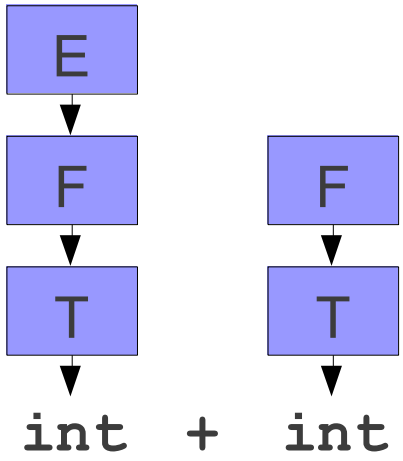
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



E + T | * int + int

A Sample Shift/Reduce Parse

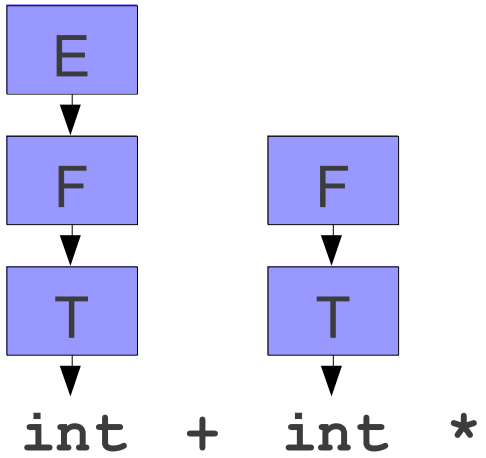
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



E + F | * int + int

A Sample Shift/Reduce Parse

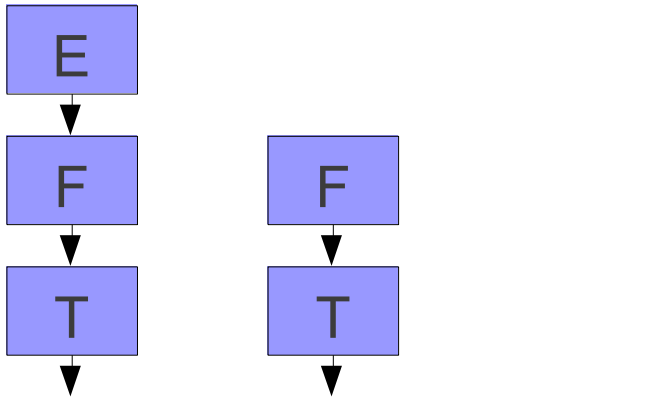
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)



E + F * | int + int

A Sample Shift/Reduce Parse

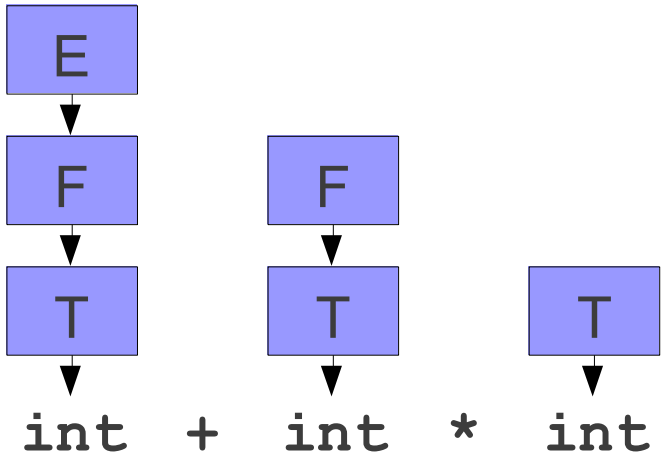
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

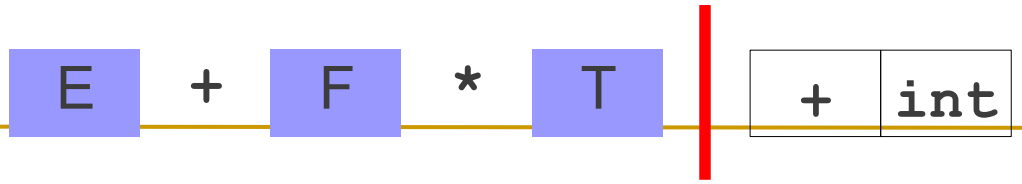


E + F * int | + int

A Sample Shift/Reduce Parse

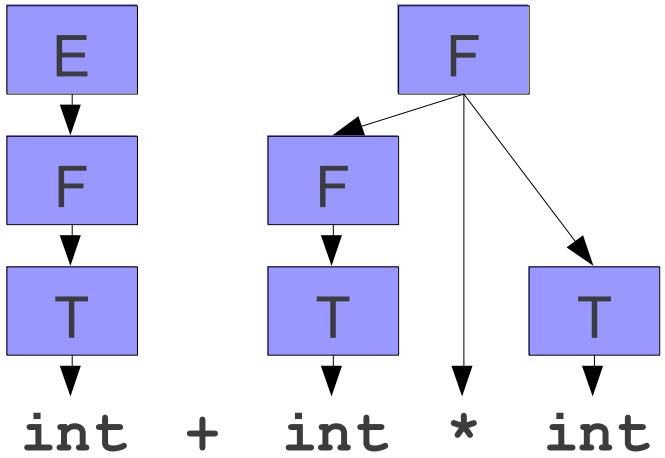
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)





A Sample Shift/Reduce Parse

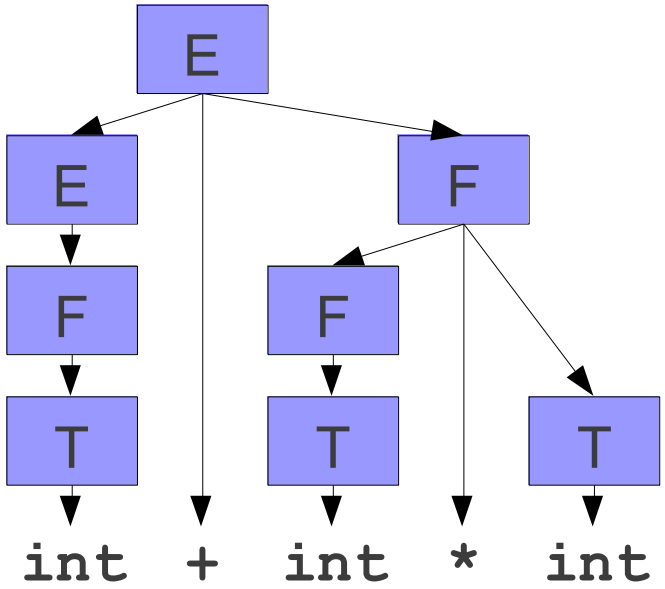
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

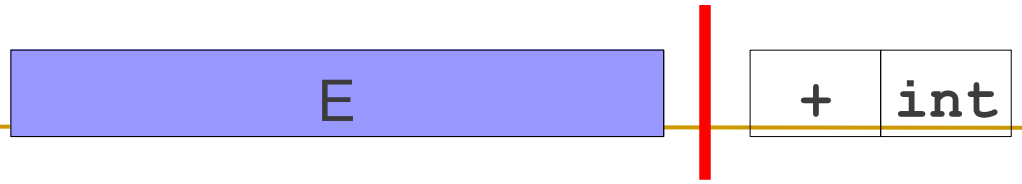




A Sample Shift/Reduce Parse

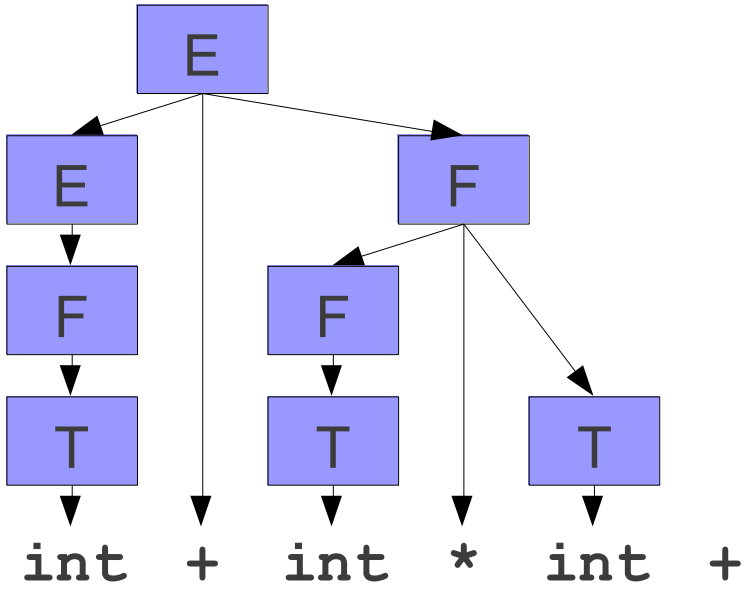
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)





A Sample Shift/Reduce Parse

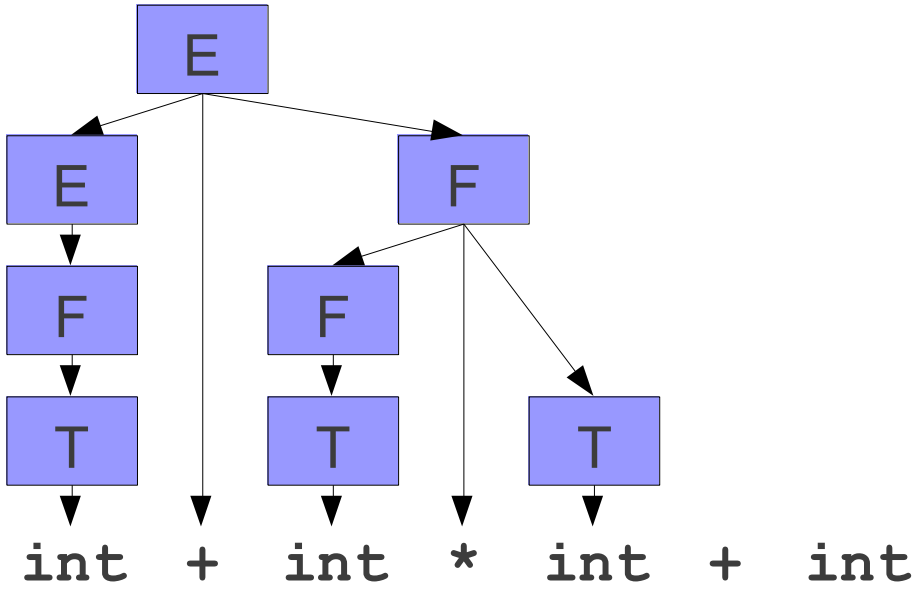
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)





A Sample Shift/Reduce Parse

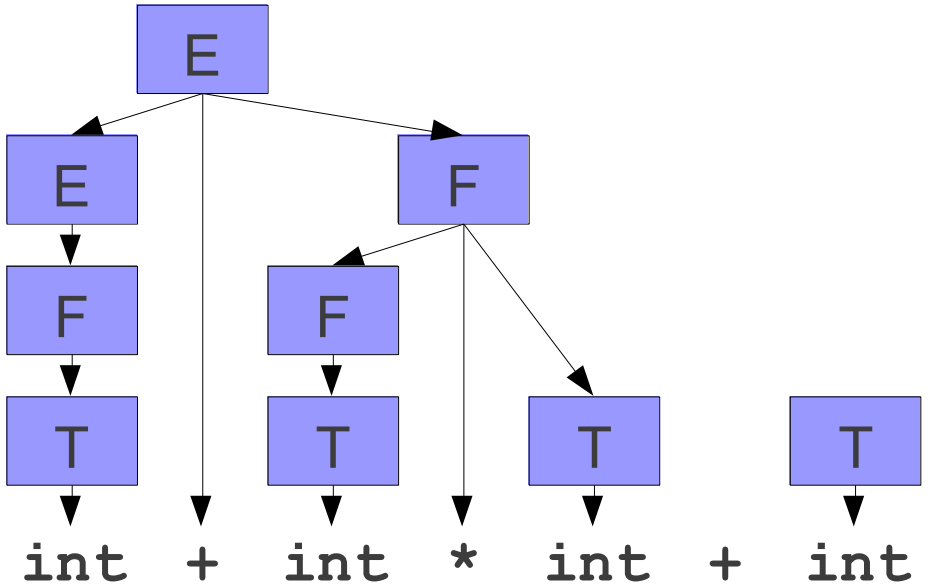
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)





A Sample Shift/Reduce Parse

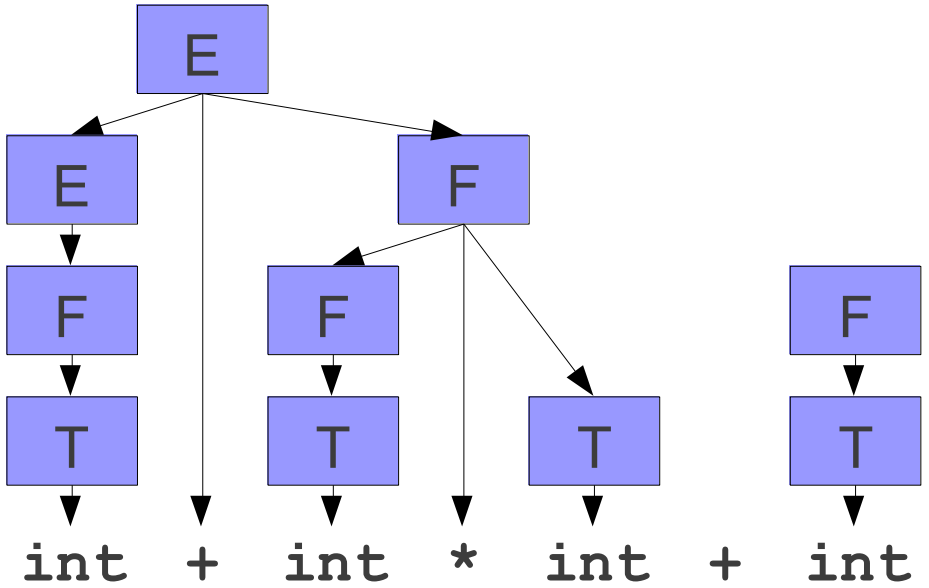
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)





A Sample Shift/Reduce Parse

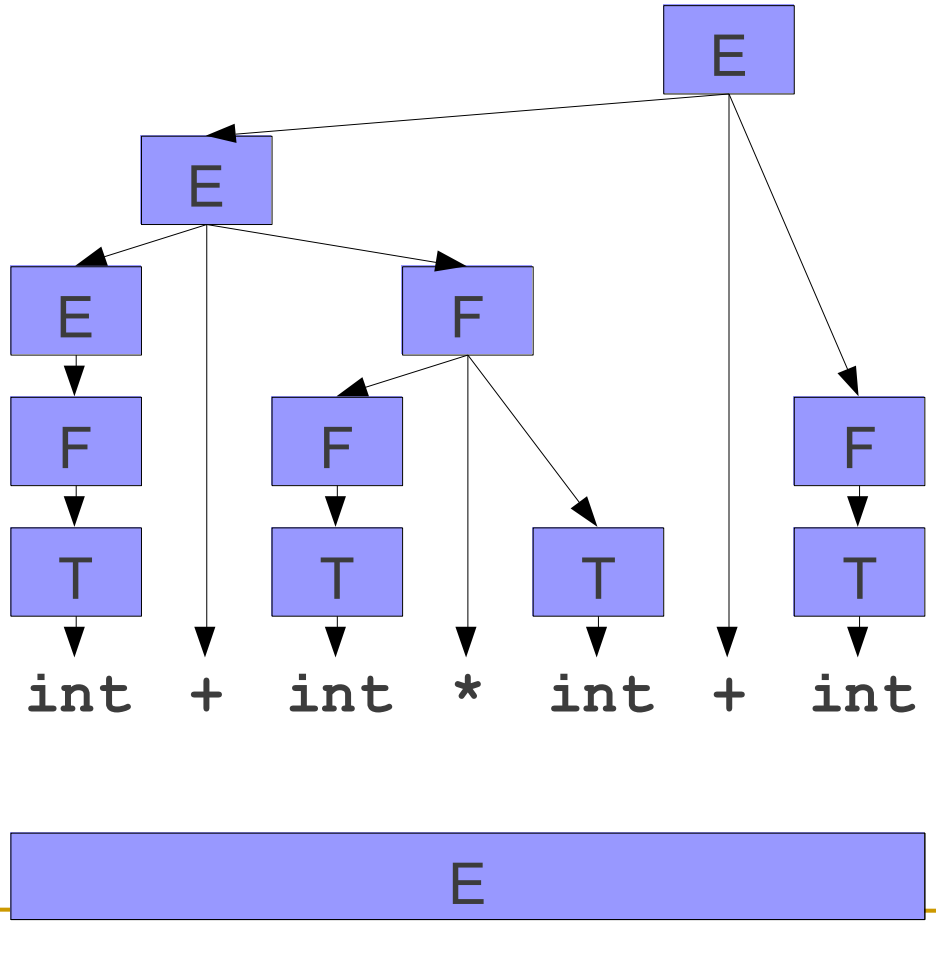
E → F
E → E + F
F → F * T
F → T
T → int
T → (E)





A Sample Shift/Reduce Parse

E → F
E → E + F
F → F * T
F → T
T → int
T → (E)

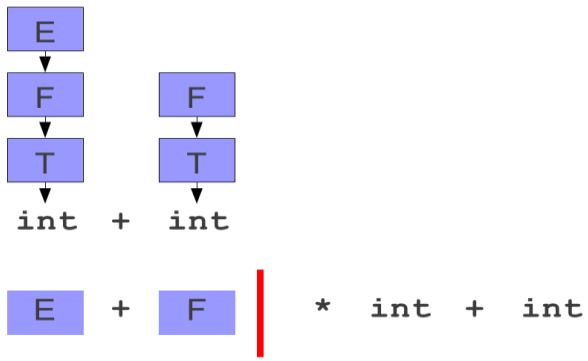


Outline

1. Intro. to bottom-up parsing
2. LR(0) parsing
    1. Intro. to LR(0) parsing
    2. LR(0) items
    3. Construct DFA from LR(0) items
    4. Shit-Reduce Parser
3. LR(1) and SLR(1) parsing

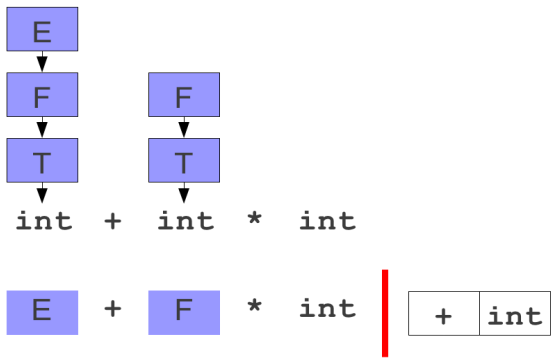
LR Parsing

$E \rightarrow F$
$E \rightarrow E + F$
$F \rightarrow F * T$
$F \rightarrow T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$

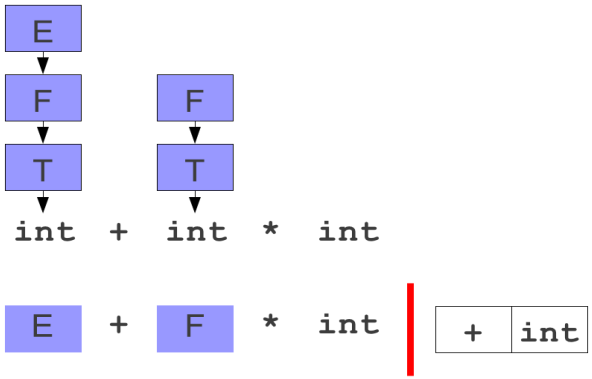


working area input area

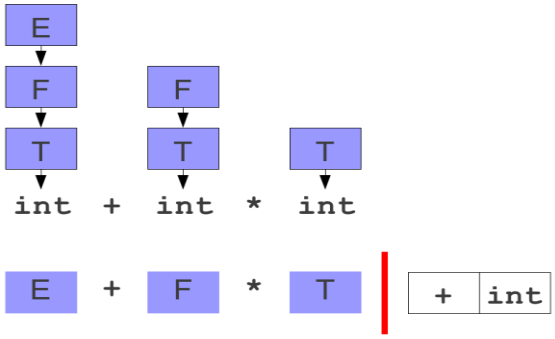
$E \rightarrow F$
$E \rightarrow E + F$
$F \rightarrow F * T$
$F \rightarrow T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$



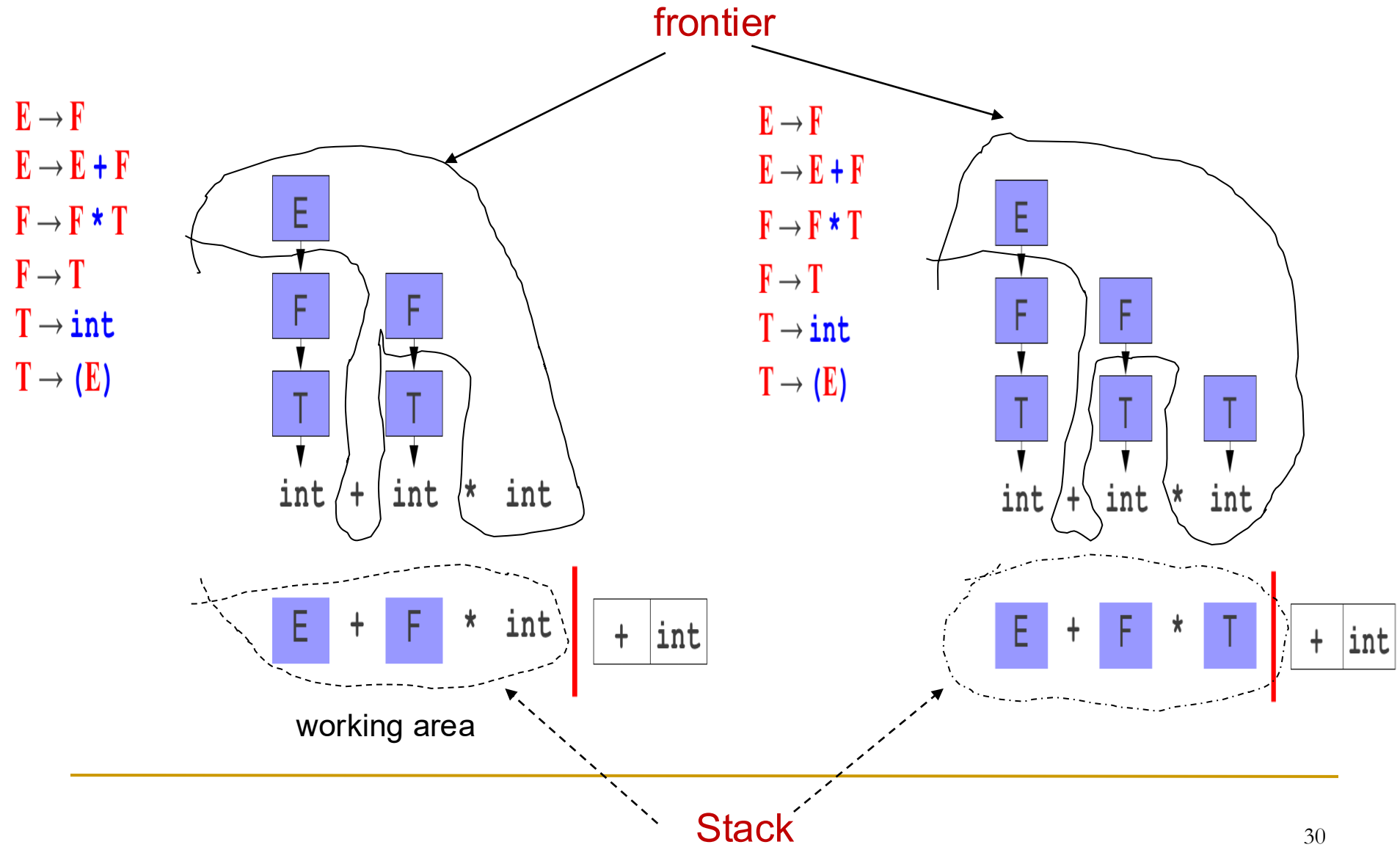
$E \rightarrow F$
$E \rightarrow E + F$
$F \rightarrow F * T$
$F \rightarrow T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$



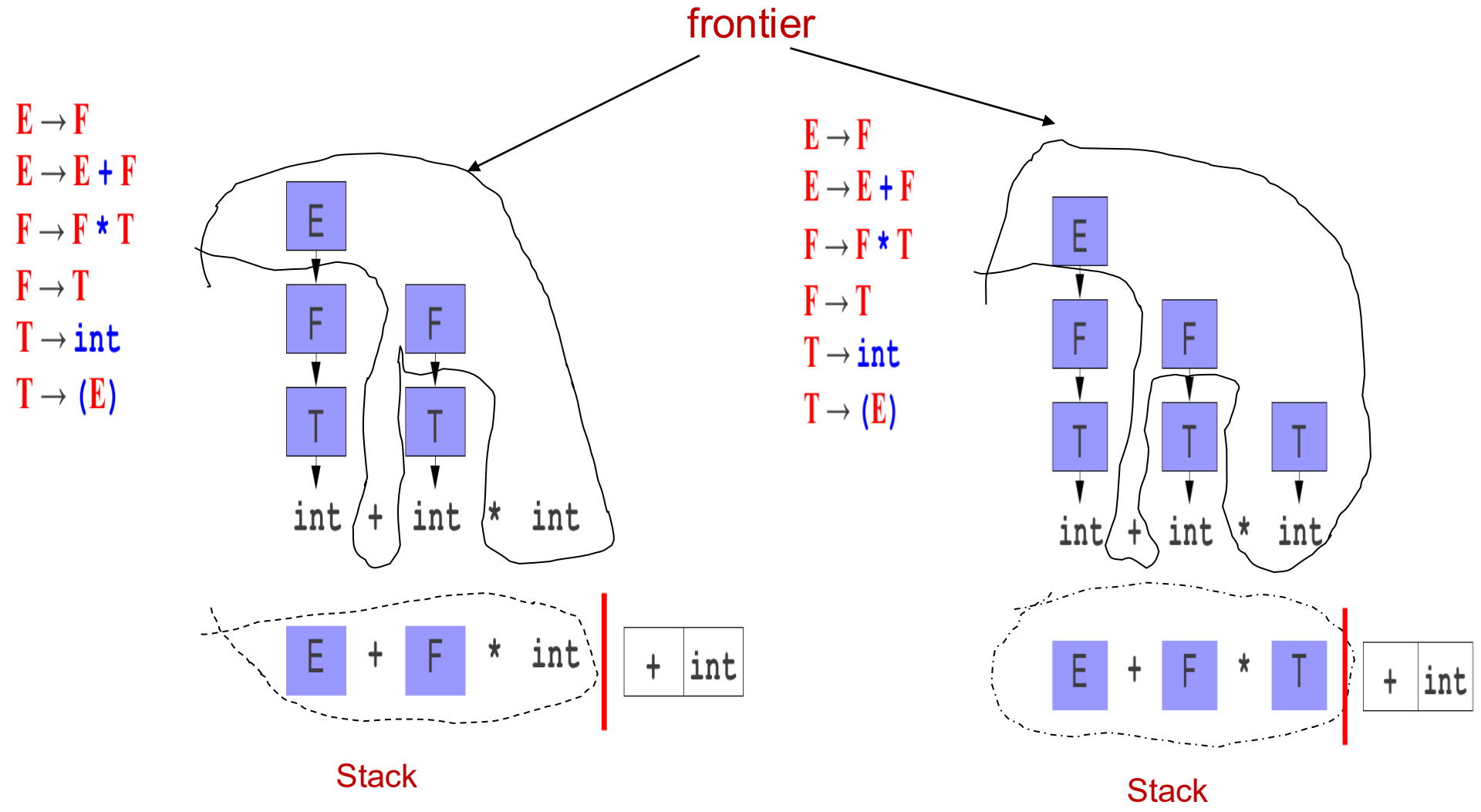
$E \rightarrow F$
$E \rightarrow E + F$
$F \rightarrow F * T$
$F \rightarrow T$
$T \rightarrow \texttt{int}$
$T \rightarrow (E)$



LR Parsing Frontier and Stack



LR Parsing Frontier and Stack



stack prefixes α guides shift/reduce actions.

LR Parser States

- Goal: know what set of reductions are legal at any given point.
- Idea: Summarize all possible stack prefixes α as a finite parser state.
  - Parser state is computed by a DFA that reads the stack σ.
  - Accept states of the DFA correspond to unique reductions that apply.

- Example: LR(0) parsing
  - **L**eft-to-right scanning, **R**ight-most derivation, **zero** look-ahead tokens
  - Too weak to handle many language grammars (e.g. the "sum" grammar)
  - But, helpful for understanding how the shift-reduce parser works.

Shift/Reduce Parsing

$S \mapsto (\ L\ ) \mid id$
$L \mapsto S \mid L, S$

- Parser state:
  - Stack of terminals and nonterminals.
  - Unconsumed input is a string of terminals
  - Current derivation step is stack + input
- Parsing is a sequence of *shift* and *reduce* operations:
- Shift: move look-ahead token to the stack: e.g.

Stack	Input	Action
	(x, (y, z), w)	shift (
(	x, (y, z), w)	shift x

- Reduce: Replace symbols γ at top of stack with nonterminal X such that $X \mapsto \gamma$ is a production. (pop γ, push X): e.g.

Stack	Input	Action
(x	, (y, z), w)	reduce $S \mapsto id$
(S	, (y, z), w)	reduce $L \mapsto S$

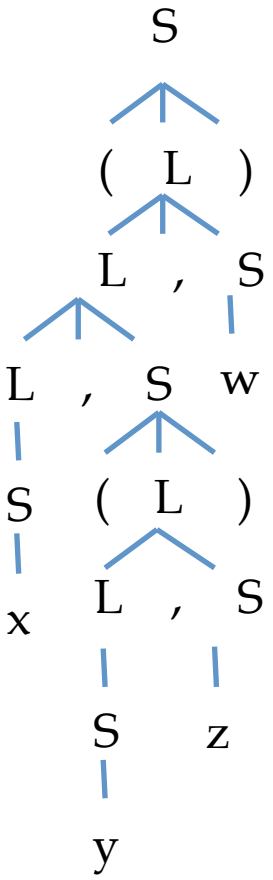
Example LR(0) Grammar: Tuples

- Example grammar for non-empty tuples and identifiers:

$$S \mapsto (\ L\ ) \mid id$$
$$L \mapsto S \mid L, S$$

- Example strings:
  - x
  - (x,y)
  - ((((x))))
  - (x, (y, z), w)
  - (x, (y, (z, w)))

Parse tree for:
(x, (y, z), w)



Example Run

$S \mapsto (L) \mid id$
$L \mapsto S \mid L , S$

Stack	Input	Action
	(x, (y, z), w)	shift (
(	x, (y, z), w)	shift x
(x	, (y, z), w)	reduce S ↦ id
(S	, (y, z), w)	reduce L ↦ S
(L	, (y, z), w)	shift ,
(L,	(y, z), w)	shift (
(L, (	y, z), w)	shift y
(L, (y	, z), w)	reduce S ↦ id
(L, (S	, z), w)	reduce L ↦ S
(L, (L	, z), w)	shift ,
(L, (L,	z), w)	shift z
(L, (L, z	), w)	reduce S ↦ id
(L, (L, S	), w)	reduce L ↦ L, S
(L, (L	), w)	shift)
(L, (L)	, w)	reduce S ↦ (L)
(L, S	, w)	reduce L ↦ L, S
(L	, w)	shift ,

Outline

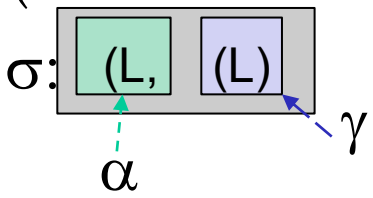
1. Intro. to bottom-up parsing
2. LR(0) parsing
    1. Intro. to LR(0) parsing
    2. LR(0) items
    3. Construct DFA from LR(0) items
    4. Shit-Reduce Parser
3. LR(1) and SLR(1) parsing

Action Selection Problem

(L, (L, z	), w)	reduce S ↦ id
(L, (L, S	), w)	reduce L ↦ L, S
(L, (L	), w)	shift)
(L, (L)	, w)	reduce S ↦ (L)

shift or reduce?

- Given a **stack** σ and a look-ahead symbol b, should the parser:
  - Shift b onto the stack (new stack is σb)
  - Reduce a production X ⟼ γ, assuming that σ = αγ (new stack is αX)?



- Sometimes the parser can reduce but shouldn't
  - For example, X ⟼ ε can *always* be reduced
- Sometimes the stack can be reduced in different ways

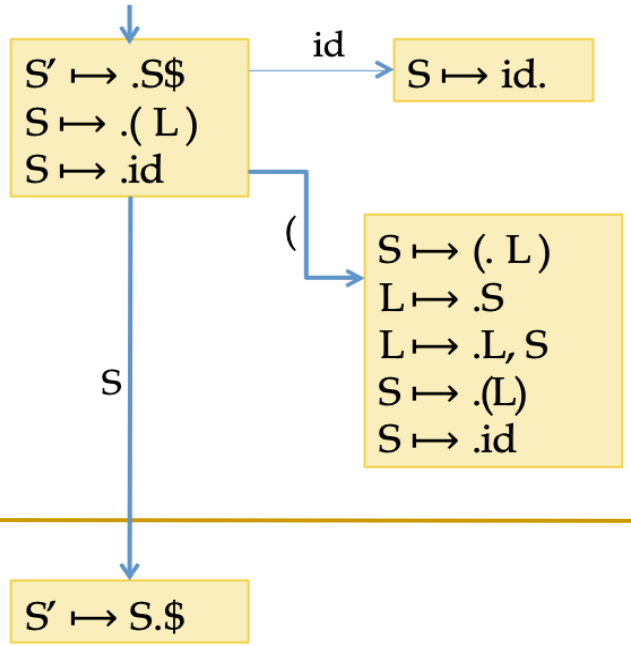
- Main idea: decide what to do based on a *prefix* α of the stack plus the look-ahead symbol.
  - The prefix α is different for different possible reductions since in productions X ⟼ γ and Y ⟼ β, the γ and β might have different lengths.

- Main goal: know what set of reductions are legal at any point.
  - How do we keep track?

Idea: Divide & Conquer

- Break down the problem to two parts:
  - parse individual grammar rule
  - parse the whole grammar
- Method:
  - Build a DFA for a grammar rule
    - The goal of the DFA is for the reduction of a rule.
    - (LR(0) states)
  - Connect DFAs of grammar rules

S ⟼ (. L)
L ⟼ .S
L ⟼ .L, S
S ⟼ .(L)
S ⟼ .id



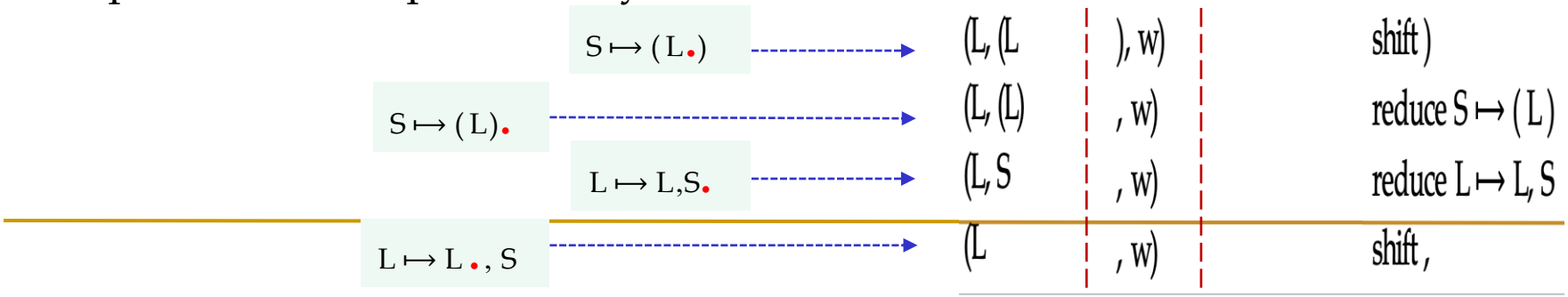
LR(0) States

- An LR(0) *state* is a *set* of *items* keeping track of progress on possible upcoming reductions.
- An LR(0) *item* is a production from the language with an extra separator "." somewhere in the right-hand-side

$$S \mapsto (\ L\ ) \mid id$$
$$L \mapsto S \mid L, S$$

- Example items: $S \mapsto .(\ L\ )$ or $S \mapsto (.\ L)$ or $L \mapsto S.$
- Intuition:
  - Stuff before the '.' is already on the stack (beginnings of possible γ's to be reduced)
  - Stuff after the '.' is what might be seen next
  - The prefixes α are represented by the state itself

Stack: $\sigma = \alpha\gamma$



LR(0) items

- LR(0) items which represent the possible states in a parse

  $[A \rightarrow \alpha \bullet \beta]$

  - α is the sequence of symbols we have seen
  - β is the sequence of symbols we expect to see
  - The prefixes α are represented by the state itself

Constructing LR(0) Items

$S \mapsto (\ L\ )$
$L \mapsto .S$
$L \mapsto .L, S$
$S \mapsto .(L)$
$S \mapsto .id$

I0 = closure({S' → •S$})

C = {I0}; set I0 unchecked

repeat

  ∀ unchecked I∈C and ∀ x ∈ (N ∪ T)

    I1 = goto(I,x)

    if I1 is a new set

      set I1 unchecked

      add I1 to C

  mark I checked

until all item sets in C are checked

goto(I,x)

  [A → α•xβ] in I

  find the item set J including

  [A → αx•β]

  return closure(J)

closure(I)

  repeat

    ∀ items [A → α•Bβ] in I

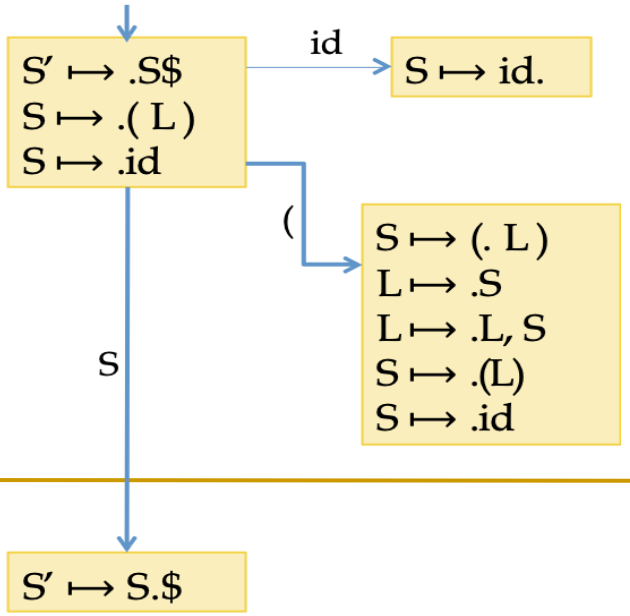
      ∀ production B →γ in G

        if [B → •γ] ∉I then

          add [B → •γ] to I

  until I does not change

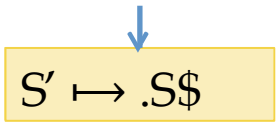
  return I



Outline

1. Intro. to bottom-up parsing
2. LR(0) parsing
    1. Intro. to LR(0) parsing
    2. LR(0) items
    3. Construct DFA from LR(0) items
    4. Shift-Reduce Parser
3. LR(1) and SLR(1) parsing

Example: Constructing the DFA



S′ ⟼ S$
S ⟼ (L) | id
L ⟼ S | L , S

- First, we construct a state with the initial item S′ ⟼ .S$

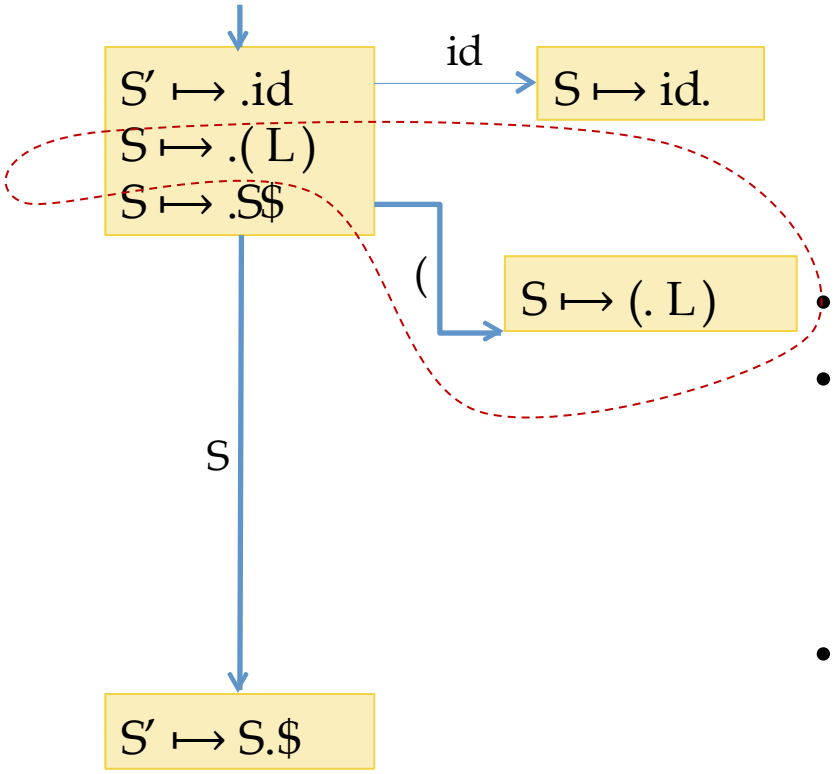
Example: Constructing the DFA

$S' \mapsto S\$$
$S \mapsto (\ L\ )\ \ |id$
$L \mapsto S\ \ |\ \ L\ ,\ S$

$S' \mapsto .S\$$
$S \mapsto .(\ L\ )$
$S \mapsto .id$

- Next, we take the closure of that state:
  CLOSURE($\{S' \mapsto .S\$\}$) = $\{S' \mapsto .S\$, S \mapsto .(\ L\ ), S \mapsto .id\}$
  - In the set of items, the nonterminal S appears after the '.'
    - $S' \mapsto .S\$$
  - So we add items for each S production in the grammar
    - $S \mapsto .(\ L\ ), S \mapsto .id$

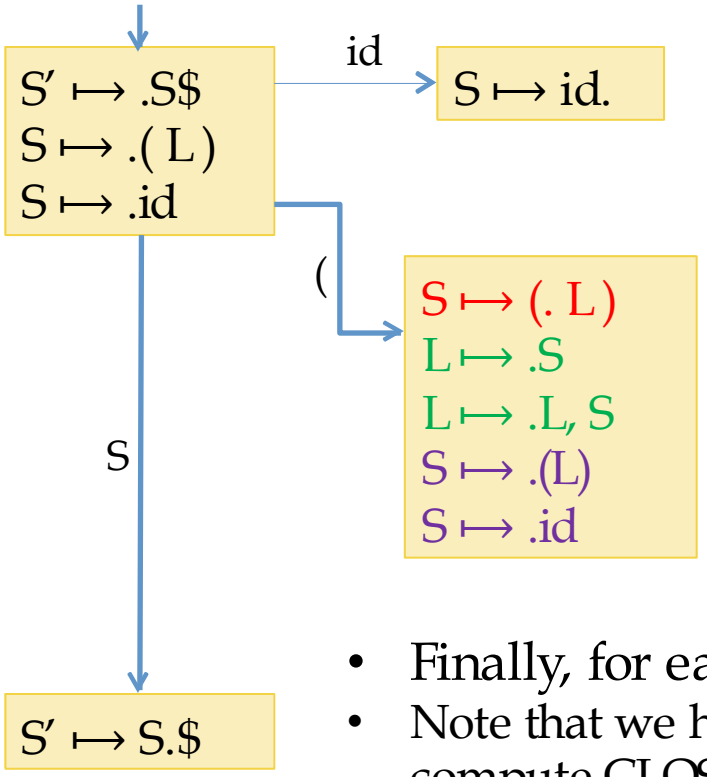
Example: Constructing the DFA



S′ ⟼ S$
S ⟼ (L) | id
L ⟼ S | L , S

- Next we add the transitions:
- First, we see what terminals and nonterminals can appear after the '.' in the source state.
  - Outgoing edges have those label.
- The target state (initially) includes all items from the source state that have the edge-label symbol after the '.', but we advance the '.' (to simulate shifting the item onto the stack)

Example: Constructing the DFA



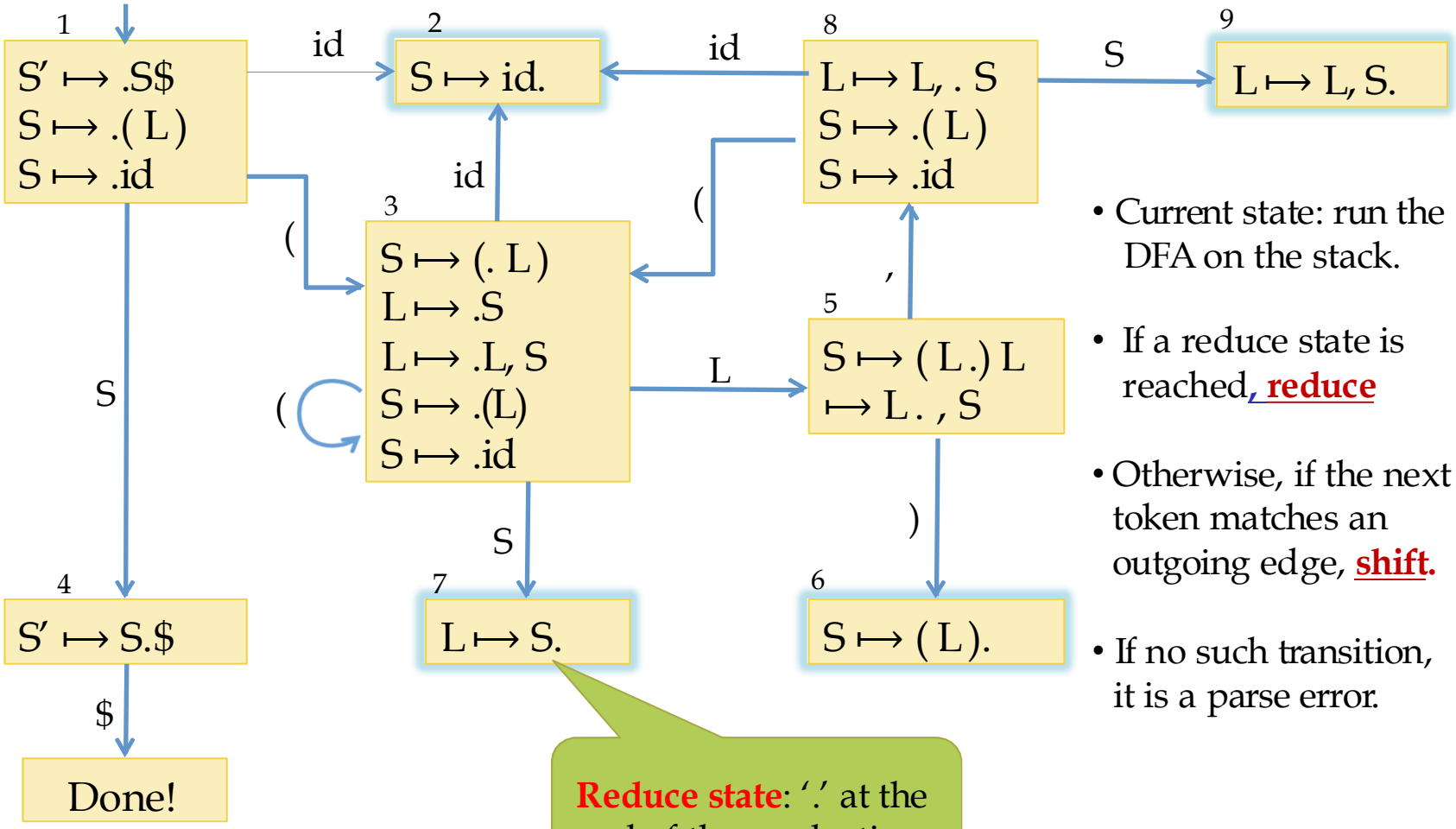
S′ ⟼ S$
S ⟼ (L) | id
L ⟼ S | L , S

Question: Why do we take the closure?

We want to see a terminal.

- Finally, for each new state, we take the closure.
- Note that we have to perform two iterations to compute CLOSURE({S ⟼ (. L)})
  - First iteration adds L ⟼ .S and L ⟼ .L, S
  - Second iteration adds S ⟼ .(L) and S ⟼ .id

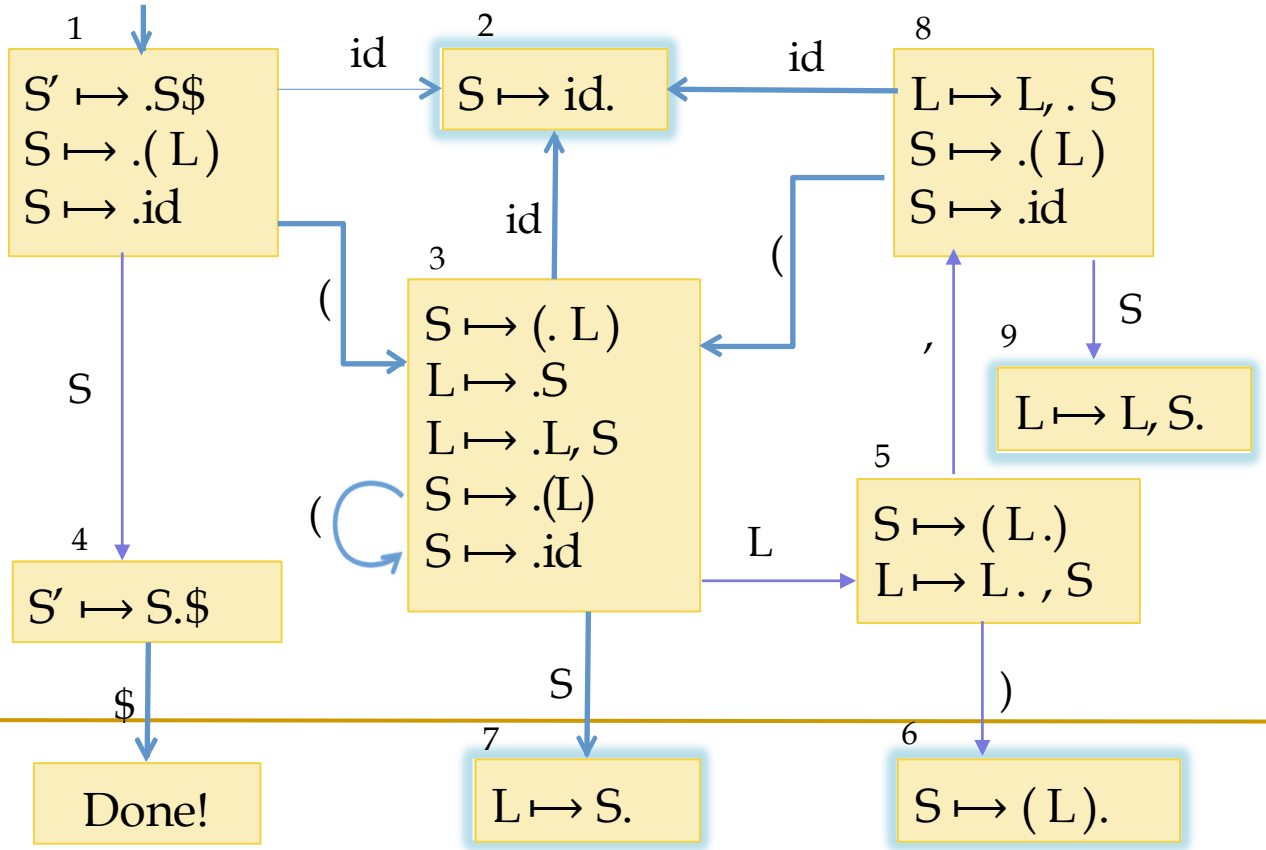
Full DFA for the Example



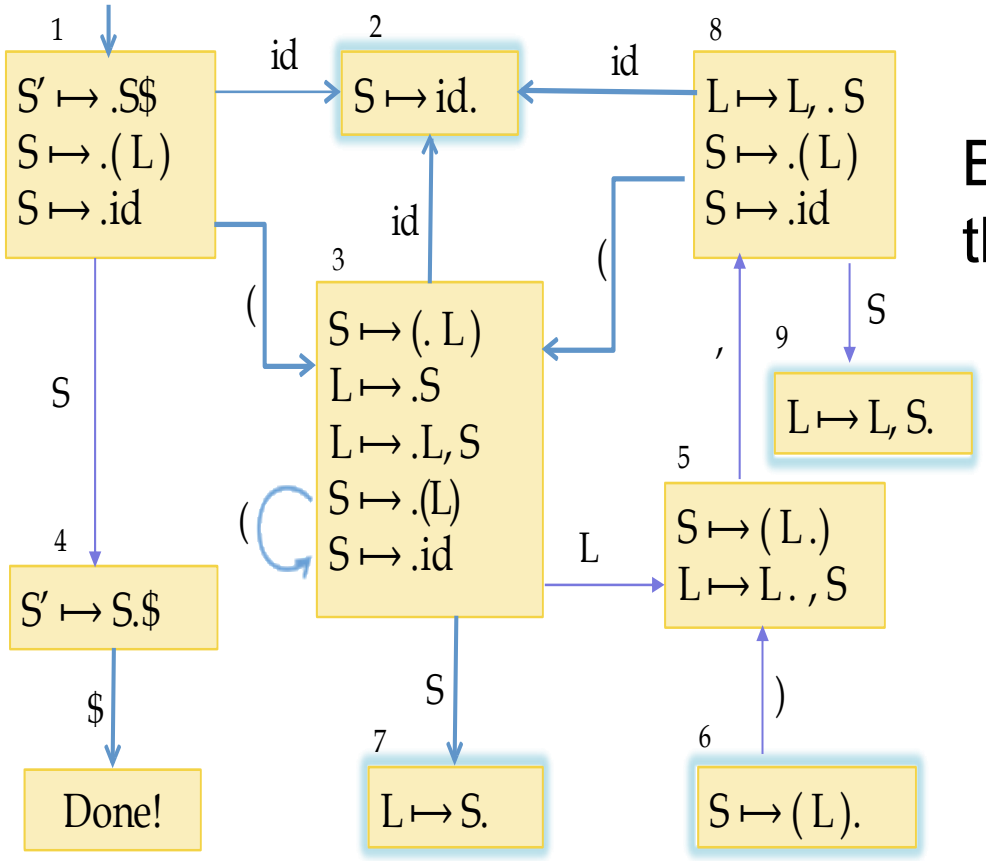
- Current state: run the DFA on the stack.
- If a reduce state is reached, **reduce**
- Otherwise, if the next token matches an outgoing edge, **shift.**
- If no such transition, it is a parse error.

Using the DFA

- Run the parser stack through the DFA.
- The resulting state tells us which productions might be reduced next.
  - If not in a reduce state, then **shift** the next symbol and transition according to DFA.
  - If in a reduce state, $X \mapsto \gamma$ with stack $\alpha\gamma$, pop γ and push X.



Using Stack to implement DFA



$1\,(_3\,(_3\,L_5\,)_6$

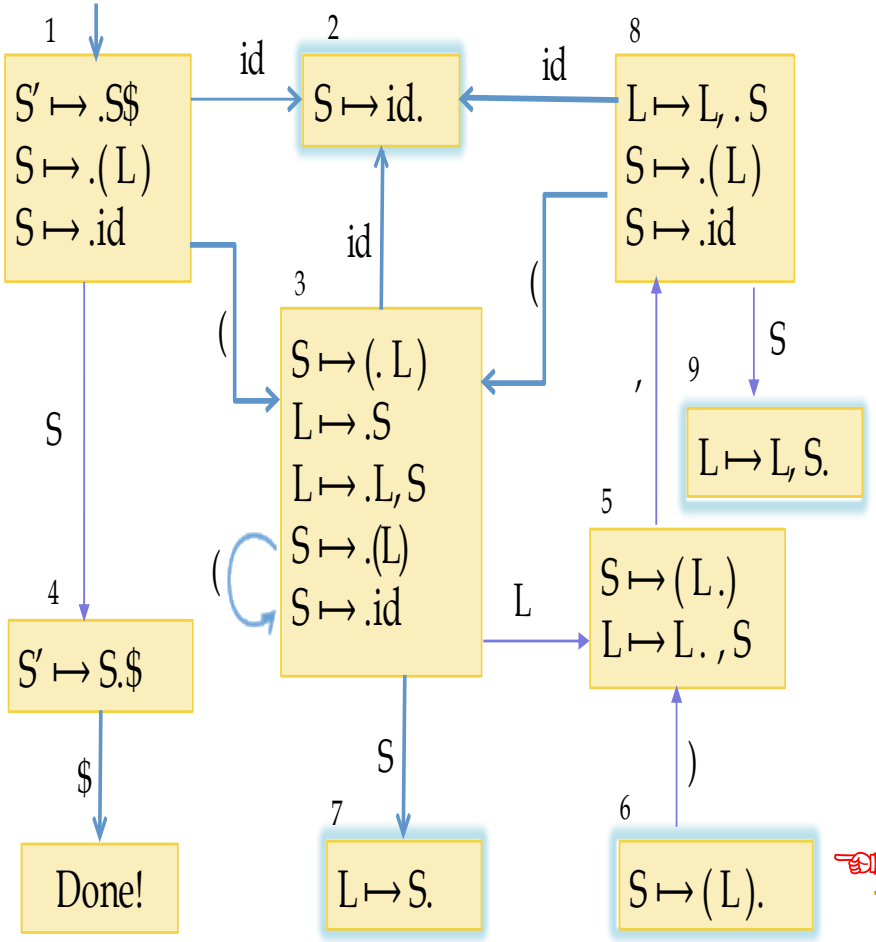
Both states and symbols are in the parsing stack.

$1\,(_3\,(_3\,L_5\,)_6$

parsing stack

Optimization

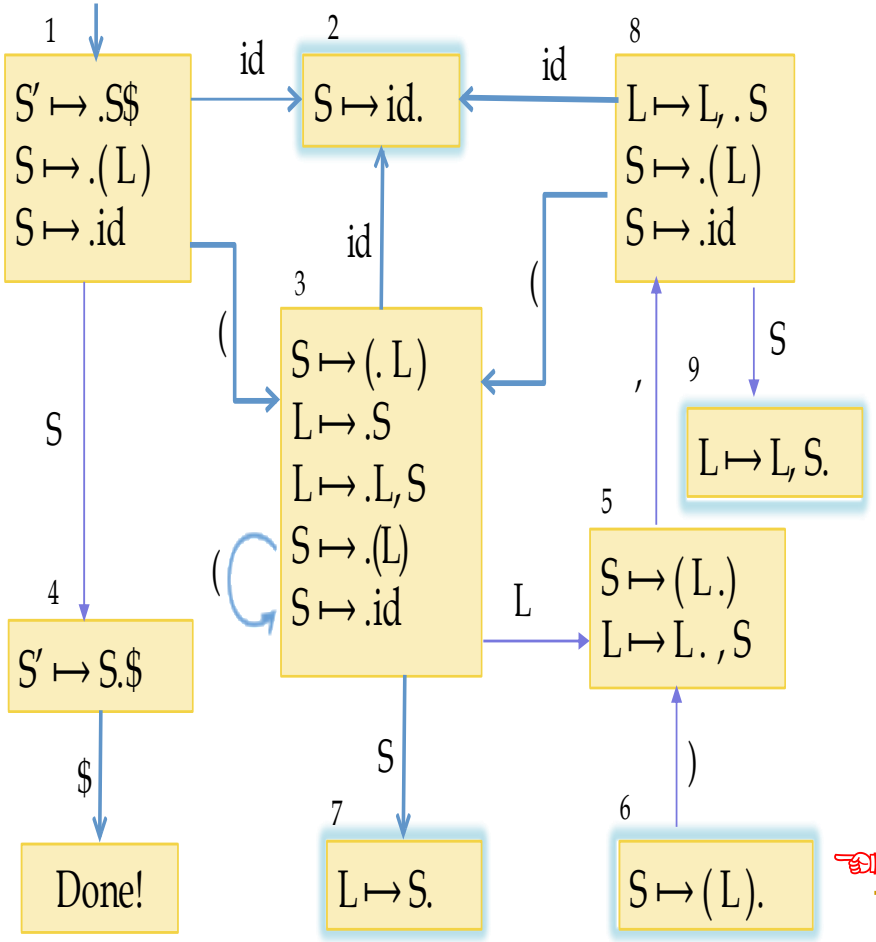
Example: (L) is reduced to S



- Optimization: No need to re-run the DFA from beginning every step
  - **Store the state with each symbol on the stack**: e.g. $1(_3(_3L_5)_6$
  -

Optimization

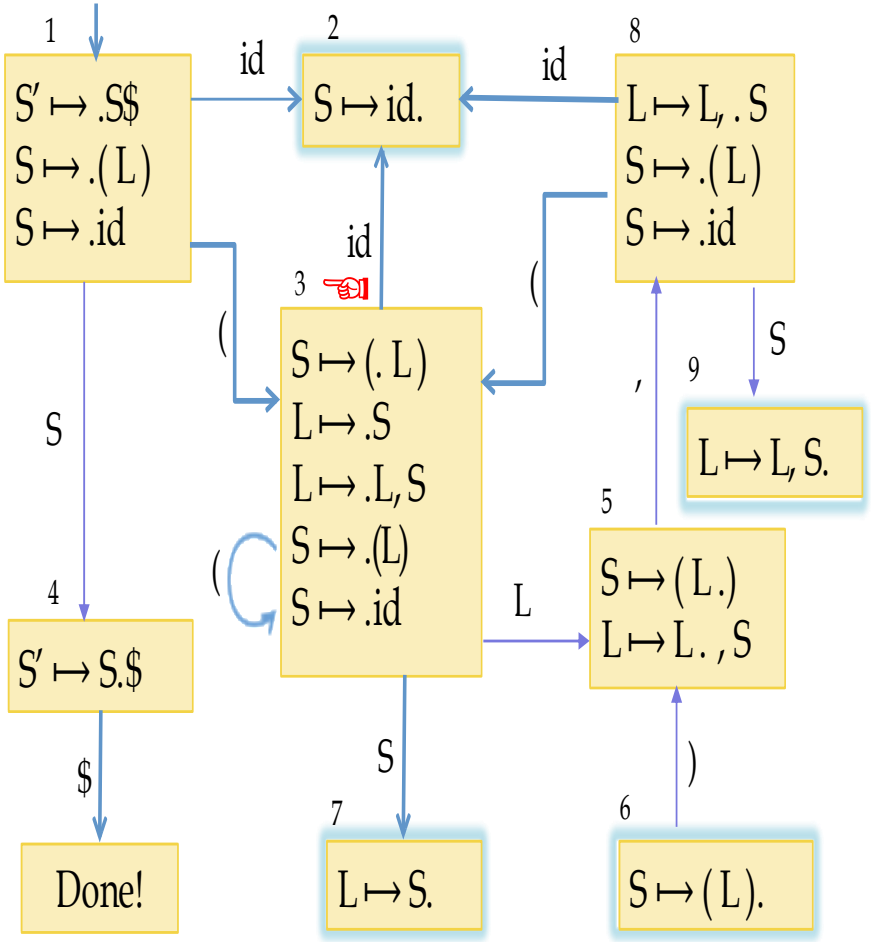
Example: (L) is reduced to S



- Optimization: No need to re-run the DFA from beginning every step
  - **Store the state with each symbol on the stack**:
    e.g. $_1(_3(_3L_5)_6$
  - On a reduction $X \longmapsto \gamma$, pop stack to reveal the state too:
    - $(_3L_5)_6$ are popped in the above example
      $_1(_3$ is left, the state 3 is shown.

Optimization

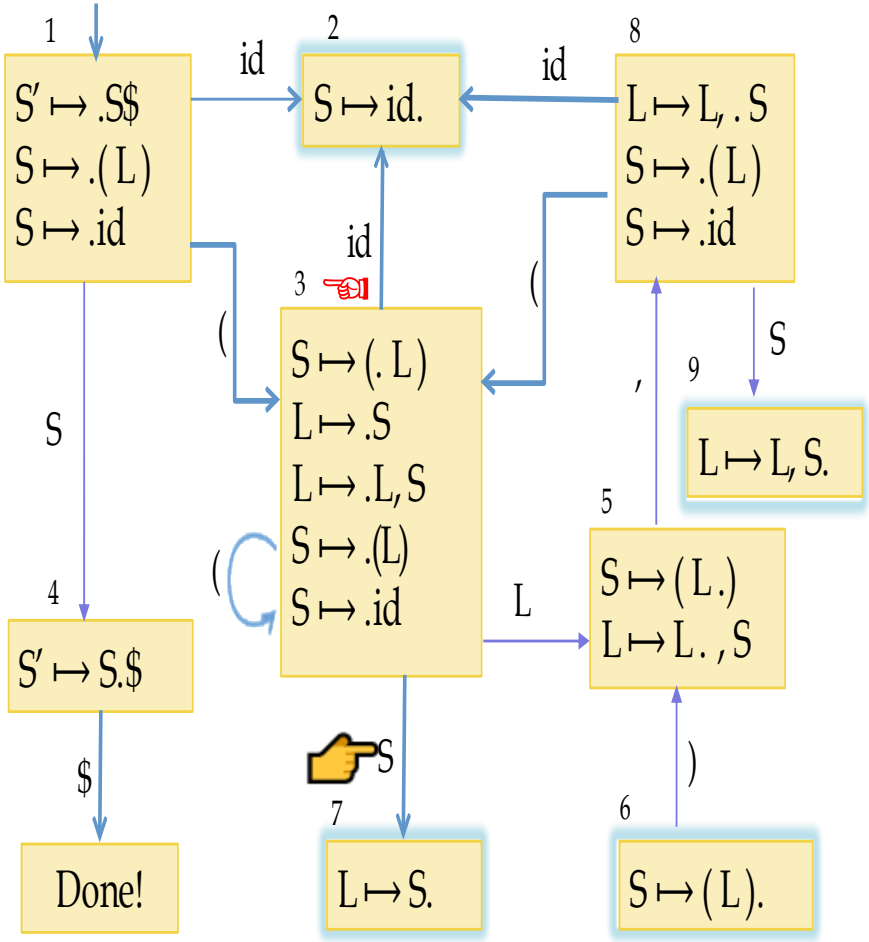
Example: (L) is reduced to S



- Optimization: No need to re-run the DFA from beginning every step
  - **Store the state with each symbol on the stack**:
    e.g. $_1(_3(_3L_5)_6$
  - On a reduction $X \longmapsto \gamma$, pop stack to reveal the state too:
    - $(_3L_5)_6$ are popped in the above example
      $_1(_3$ is left, the state 3 is shown.
    - Next, push the reduction symbol, S,. to reach stack $_1(_3S$

Optimization

Example: (L) is reduced to S



- Optimization: No need to re-run the DFA from beginning every step
  - **Store the state with each symbol on the stack**:
    e.g. $_1(_3(_3L_5)_6$
  - On a reduction $X \mapsto \gamma$, pop stack to reveal the state too:
    - $(_3L_5)_6$ are popped in the above example
      $_1(_3$ is left, the state 3 is shown.
    - Next, push the reduction symbol, S,. to reach stack $_1(_3S$
    - Then take just one step in the DFA to find next state: $_1(_3S_7$

Build the Parsing Table

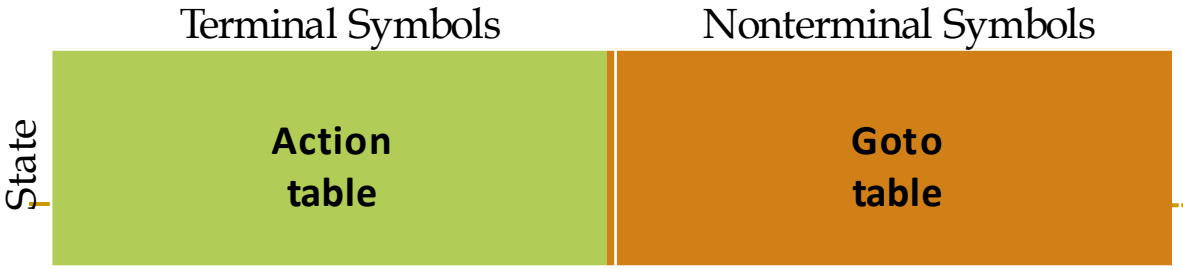
Represent the DFA as a table of shape:

state * (terminals + nonterminals)

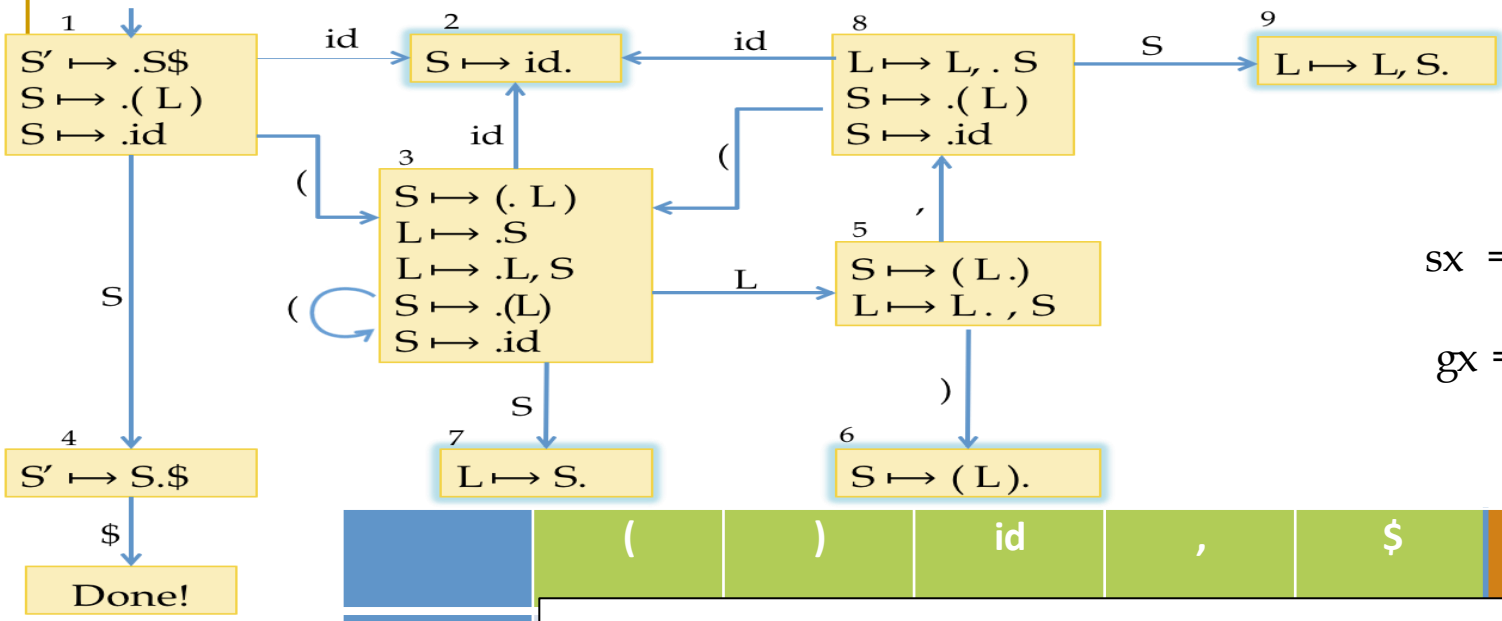
- Entries for the "action table" specify two kinds of actions:
  - Shift and goto state n
  - Reduce using reduction $X \longmapsto \gamma$
    - First pop γ off the stack to reveal the state s
    - Push X to the stack
    - Look up X in the "goto table" for the state s and goto the target state

  - **Store the state with each symbol on the stack**:
    e.g. $_1(_3(L_5)_6$
  - On a reduction $X \longmapsto \gamma$, pop stack to reveal the state too:
    - $(_3L_5)_6$ are popped in the above example
      $_1(_3$ is left, the state 3 is shown
    - Next, push the reduction symbol, S,. to reach stack $_1(_3S$
    - Then take just one step in the DFA to find next state: $_1(_3S_7$

- Missing table entries = parse errors



Example: Build Parse Table

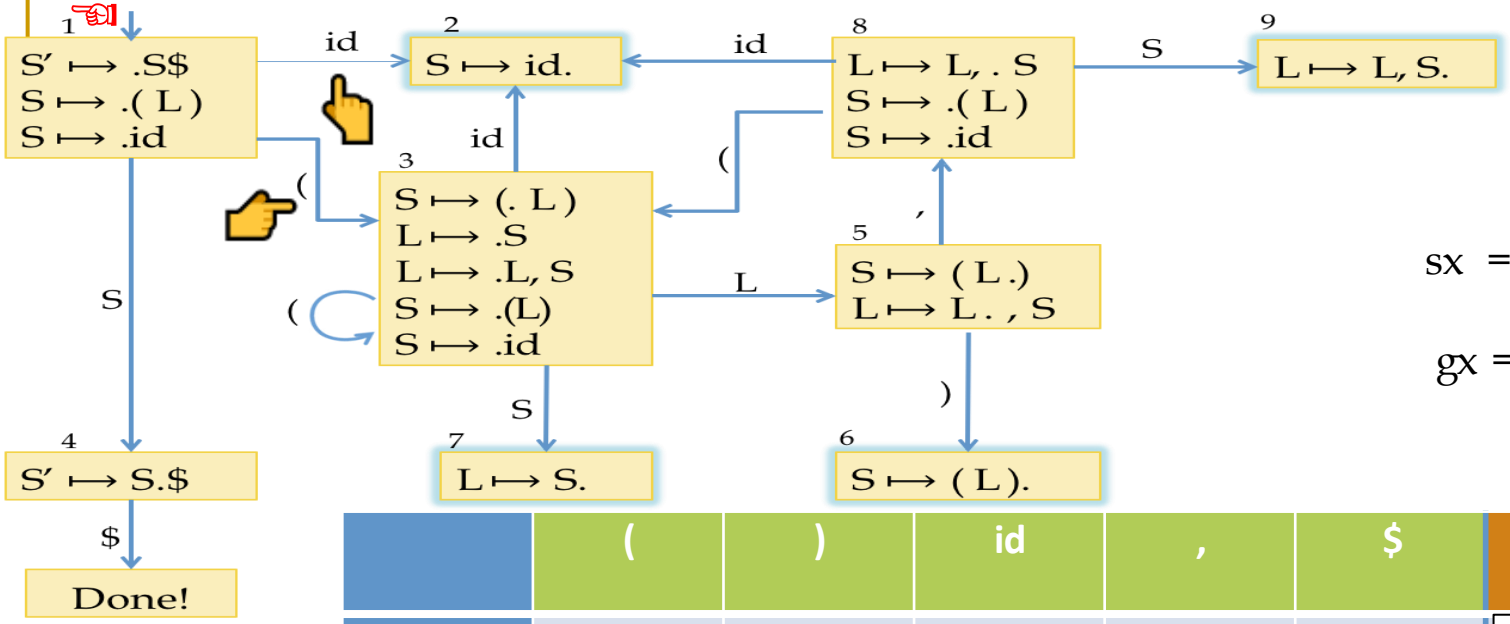


sx = shift and goto state x

gx = goto state x

	(	)	id	,	$	S	L
1							
2							
3							
4							
5							
6							
7							
8							
9							

Example: Build Parse Table

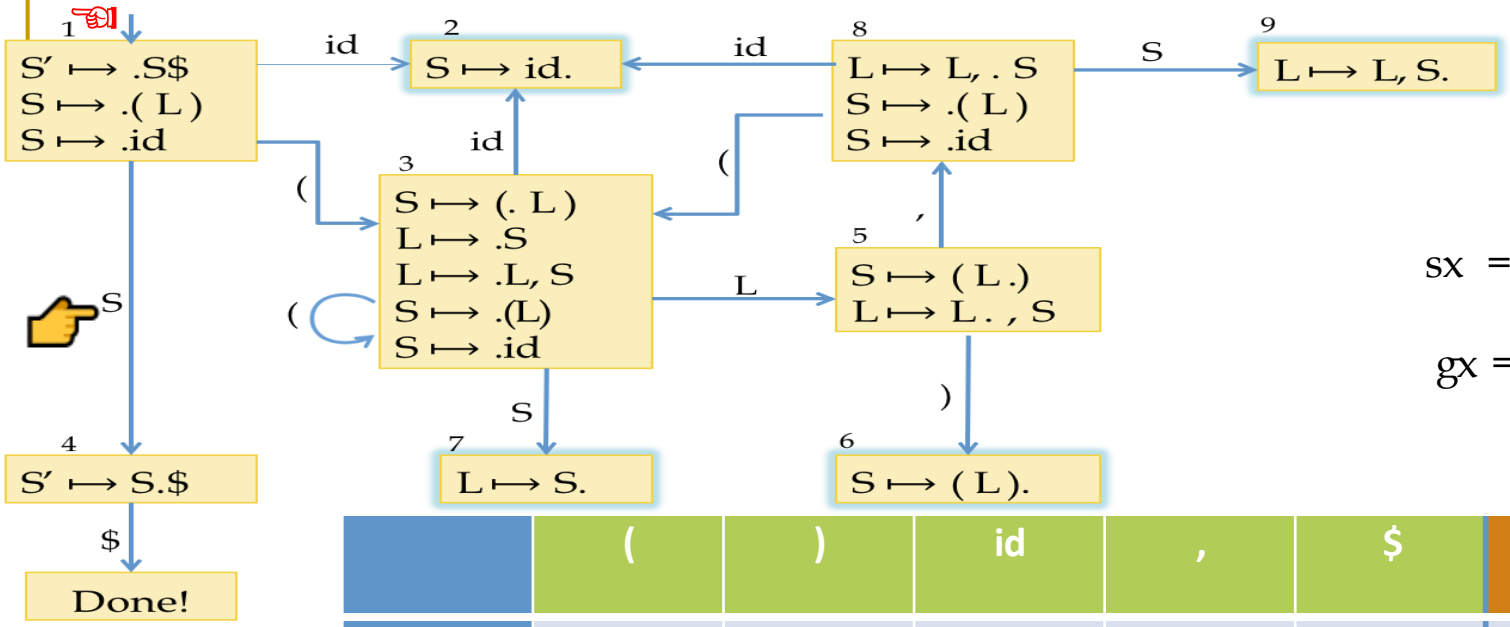


sx = shift and goto state x

gx = goto state x

	(	)	id	,	$	S	L
1	s3		s2				
2							
3							
4							
5							
6							
7							
8							
9							

Example: Build Parse Table

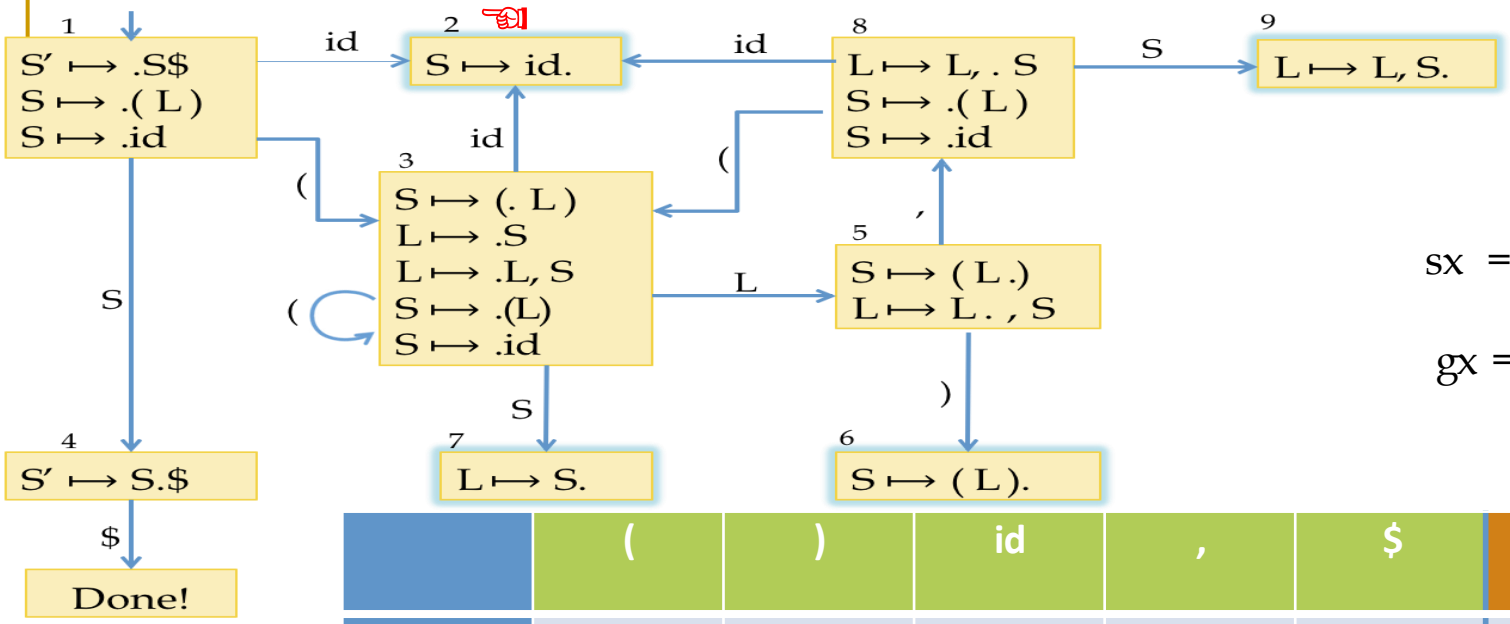


sx = shift and goto state x

gx = goto state x

	(	)	id	,	$	S	L
1	s3		s2			g4	
2							
3							
4							
5							
6							
7							
8							
9							

Example: Build Parse Table



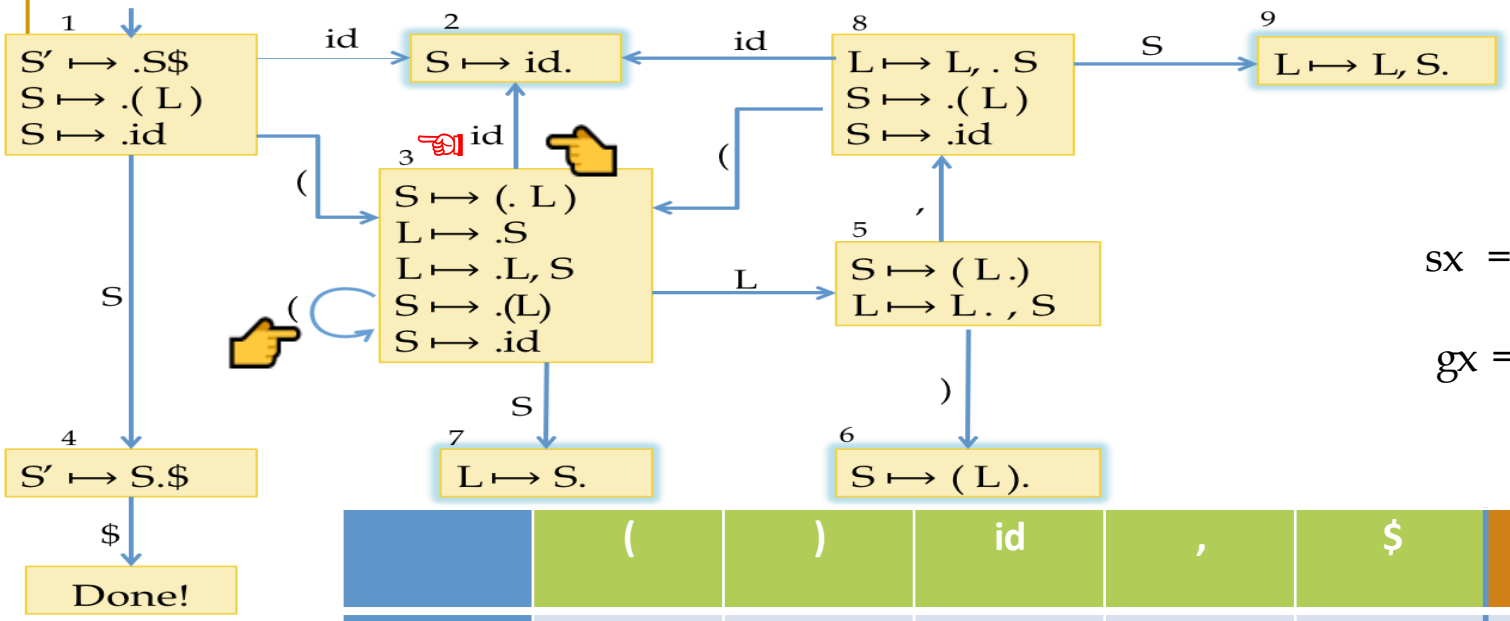
sx = shift and goto state x

gx = goto state x

	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S⟼id	S⟼id	S⟼id	S⟼id	S⟼id		
3							
4							
5							
6							
7							
8							
9							

State 2
Is a reduce state.

Example: Build Parse Table

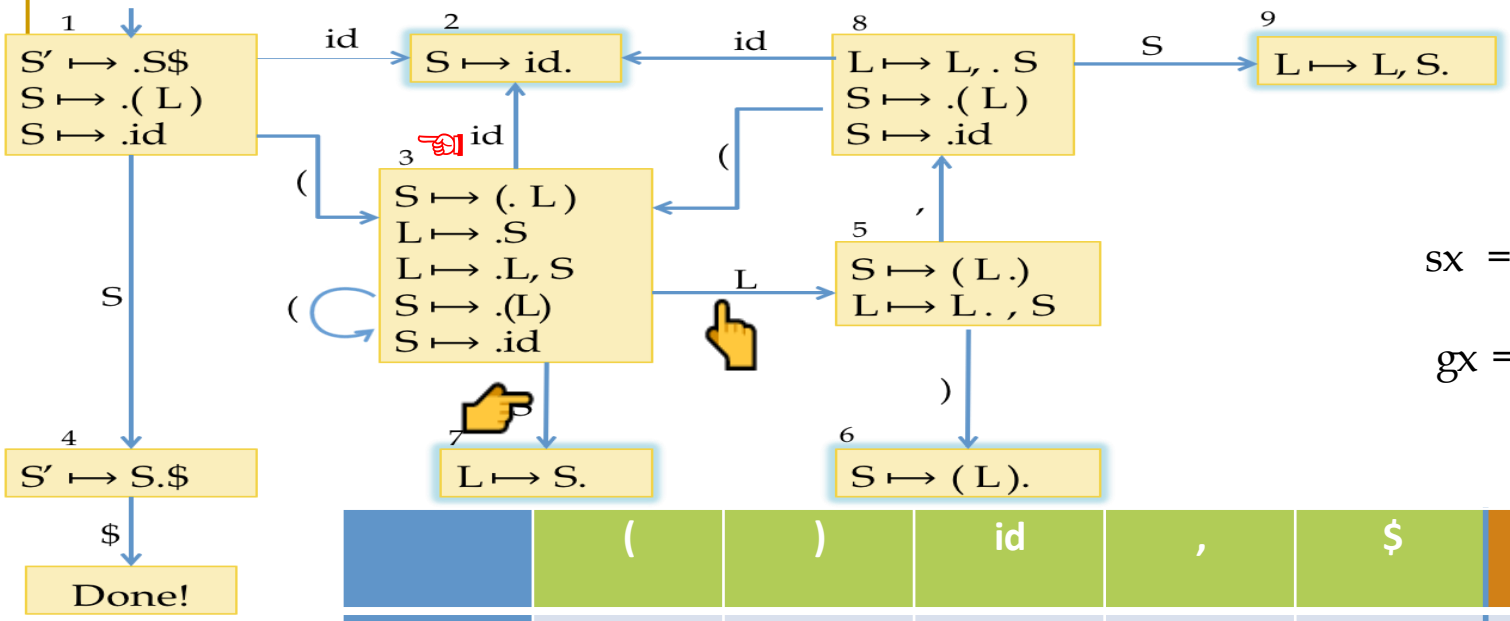


sx = shift and goto state x

gx = goto state x

	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S⟼id	S⟼id	S⟼id	S⟼id	S⟼id		
3	s3		s2				
4							
5							
6							
7							
8							
9							

Example: Build Parse Table

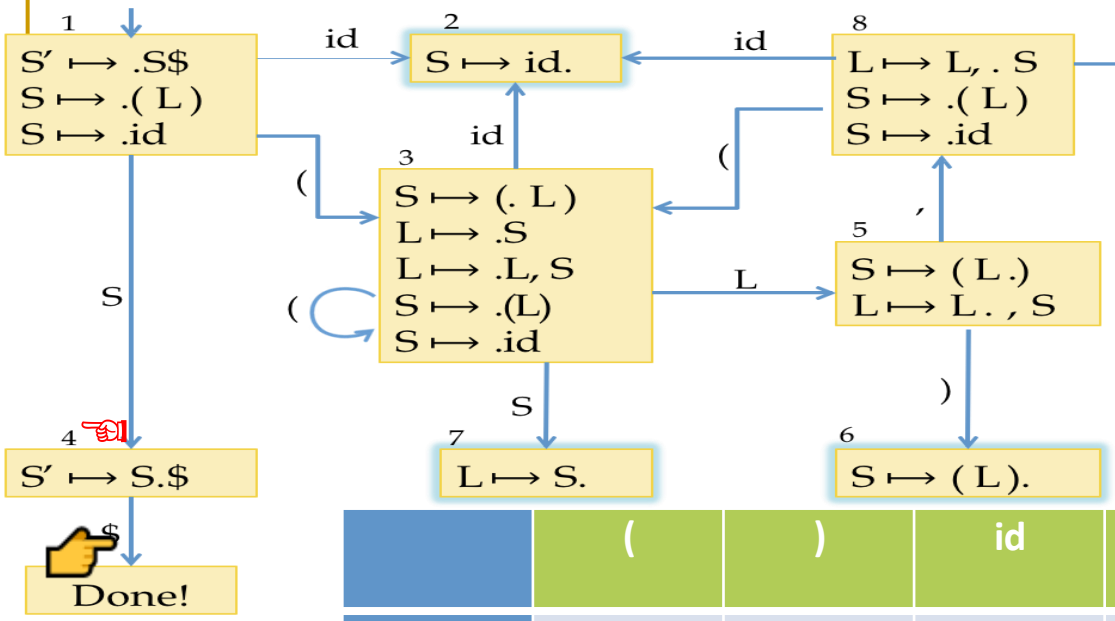


sx = shift and goto state x

gx = goto state x

	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S⟼id	S⟼id	S⟼id	S⟼id	S⟼id		
3	s3		s2			g7	g5
4							
5							
6							
7							
8							
9							

In class exercise : Finish the Parse Table

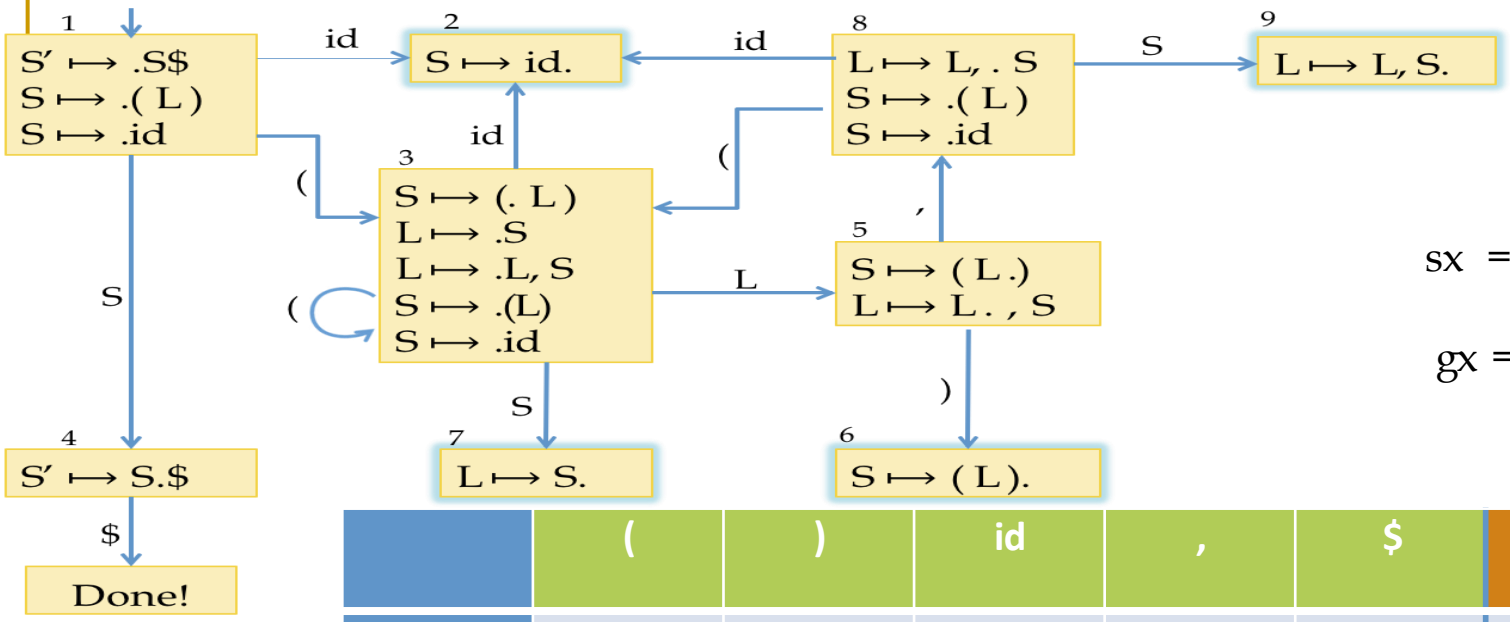


sx = shift and goto state x

gx = goto state x

	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S⟼id	S⟼id	S⟼id	S⟼id	S⟼id		
3	s3		s2			g7	g5
4					DONE		
5							
6							
7							
8							
9							

Example: Build Parse Table



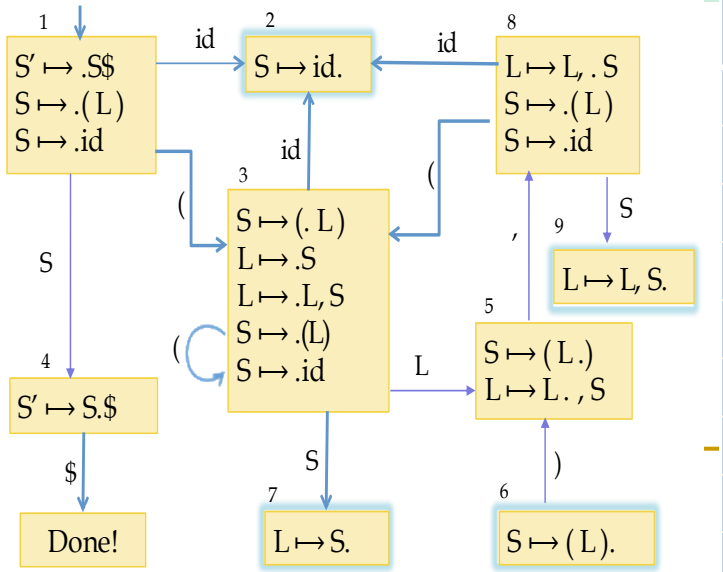
sx = shift and goto state x

gx = goto state x

	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S⟼id	S⟼id	S⟼id	S⟼id	S⟼id		
3	s3		s2			g7	g5
4					DONE		
5		s6		s8			
6	S ⟼ (L)	S ⟼ (L)	S ⟼ (L)	S ⟼ (L)	S ⟼ (L)		
7	L ⟼ S	L ⟼ S	L ⟼ S	L ⟼ S	L ⟼ S		
8	s3		s2			g9	
9	L ⟼ L,S	L ⟼ L,S	L ⟼ L,S	L ⟼ L,S	L ⟼ L,S		

Example Parse the token stream: (x, (y, z), w)$

Stack	Stream	Action (according to table)
ε_1	(x, (y, z), w)$	s3
$\varepsilon_1(_3$	x, (y, z), w)$	s2
$\varepsilon_1(_3x_2$	, (y, z), w)$	Reduce: S⟼id
$\varepsilon_1(_3S$	, (y, z), w)$	g7 (from state 3 follow S)
$\varepsilon_1(_3S_7$	, (y, z), w)$	Reduce: L⟼S
$\varepsilon_1(_3L$	, (y, z), w)$	g5 (from state 3 follow L)
$\varepsilon_1(_3L_5$	, (y, z), w)$	s8
$\varepsilon_1(_3L_{5,8}$	(y, z), w)$	s3
$\varepsilon_1(_3L_{5,8}(_3$	y, z), w)$	s2



	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S⟼id	S⟼id	S⟼id	S⟼id	S⟼id		
3	s3		s2			g7	g5
4					DONE		
5		s6		s8			
6	S ↦ (L)	S ↦ (L)	S ↦ (L)	S ↦ (L)	S ↦ (L)		
7	L ↦ S	L ↦ S	L ↦ S	L ↦ S	L ↦ S		
8	s3		s2			g9	
9	L ↦ L,S	L ↦ L,S	L ↦ L,S	L ↦ L,S	L ↦ L,S		

Outline

1. Intro. to bottom-up parsing
2. LR(0) parsing
    1. Intro. to LR(0) parsing
    2. LR(0) items
    3. Construct DFA from LR(0) items
    4. Shit-Reduce Parser
3. LR(1) and SLR(1) parsing

Shift-Reduce Parsers

A **handle** consists of several tokens, on the top of the stack, to be reduced.

- Bottom-up parser with 3 actions

1. *shift* - next input symbol and the state are pushed onto the stack
2. *reduce* - *pop* a handle off the stack, and *push* the lhs of the corresponding production to the stack
3. *accept* - terminate parsing and signal success

	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S↦id	S↦id	S↦id	S↦id	S↦id		
3	s3		s2			g7	g5
4					accept		
5		s6		s8			
6	S ↦ (L)	S ↦ (L)	S ↦ (L)	S ↦ (L)	S ↦ (L)		
7	L ↦ S	L ↦ S	L ↦ S	L ↦ S	L ↦ S		
8	s3		s2			g9	
9	L ↦ L,S	L ↦ L,S	L ↦ L,S	L ↦ L,S	L ↦ L,S		

Shift-Reduce Algorithm

```
Stack.Push(S1)
repeat {
  s = Stack.Top();
  a = NextToken();
  if action[s,a] == 'shift Sk' then
     Stack.Push(a); Stack.Push(Sk);
  else
     if action[s,a] == 'reduce A → α' {
              Stack.Pop(2*|α|);
              Sk = Stack.Top();
              Stack.Push(A);
              Stack.Push(goto[Sk, A]);
              Emit(A → α);
     }
  else if action[s,a] = accept then {
     Emit("Success");
     return;
  else
     error()
}
```

S1: the initial state, DONE: accept state

	(	)	id	,	$	S	L
1	s3		s2			g4	
2	S↦id	S↦id	S↦id	S↦id	S↦id		
3	s3		s2			g7	g5
4					DONE		
5		s6		s8			
6	S ↦ (L)	S ↦ (L)	S ↦ (L)	S ↦ (L)	S ↦ (L)		
7	L ↦ S	L ↦ S	L ↦ S	L ↦ S	L ↦ S		
8	s3		s2			g9	
9	L ↦ L,S	L ↦ L,S	L ↦ L,S	L ↦ L,S	L ↦ L,S		

Stack	Stream	Action (according to table) s3
ε_1	(x, (y, z), w)$	
$\varepsilon_1(_3$	x, (y, z), w)$	s2
$\varepsilon_1(_3x_2$	, (y, z), w)$	Reduce: S↦id
$\varepsilon_1(_3S$	, (y, z), w)$	g7 (from state 3 follow S)
$\varepsilon_1(_3S_7$	, (y, z), w)$	Reduce: L↦S
$\varepsilon_1(_3L$	, (y, z), w)$	g5 (from state 3 follow L)
$\varepsilon_1(_3L_5$	, (y, z), w)$	s8
$\varepsilon_1(_3L_{5,8}$	(y, z), w)$	s3
$\varepsilon_1(_3L_{5,8}(_3$	y, z), w)$	s2

Example Grammar

1. S' → E
2. E → T + E
3. | T
4. T → F * T
5. | F
6. F → num

num is a terminal defined as [0-9]+

Example Parse Table

	ACTION				GOTO		
	num	+	*	$	E	T	F
S_0	s4				1	2	3
S_1				!			
S_2		s5		r3			
S_3		r5	s6	r5			
S_4		r6	r6	r6			
S_5	s4				7	2	3
S_6	s4					8	3
S_7				r2			
S_8		r4		r4			

shift, goto S_6

reduce with rule #5 (rules listed in slide 67)

Outline

1. Intro. to bottom-up parsing
2. LR(0) parsing
3. SLR(1) parsing

LR(0) Limitations

- An LR(0) machine only works if states with reduce actions have a *single* reduce action.
  - In such states, the machine *always* reduces (ignoring lookahead)
- With more complex grammars, the DFA construction will yield states with shift/reduce and reduce/reduce conflicts:

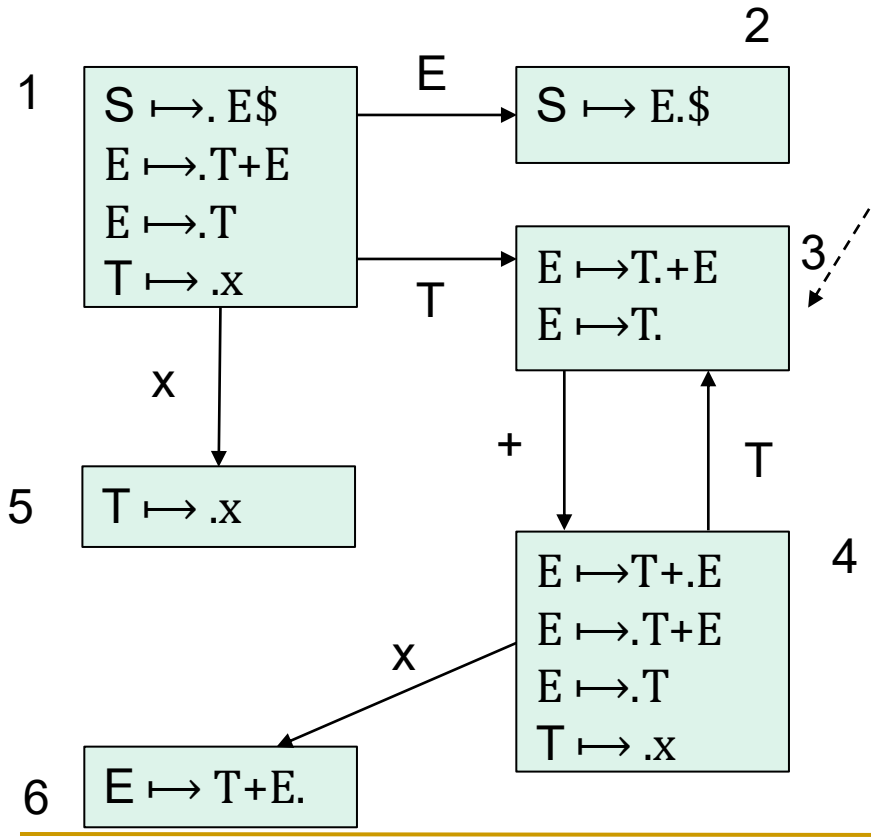
OK	shift/reduce	reduce/reduce
S ⟼ (L).	S ⟼ (L). L ⟼ .L , S	S ⟼ L ,S. S ⟼ ,S.

  - Such conflicts can often be resolved by using a look-ahead symbol: LR(1)

Example

Grammar

(0) S ⟼ E $, (1) E ⟼ T + E, (2) E ⟼ T, (3) T ⟼ x.



Conflict

	x	+	$	E	T
1	s5			g2	g3
2			a		
3	r2	s4,r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		

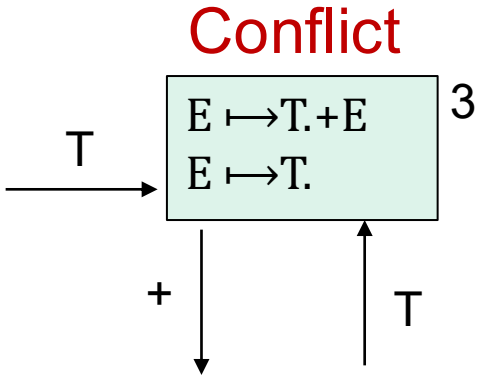
How to handle the LR(0) conflicts?

Grammar

(0) S ⟼ E $, (1) E ⟼ T + E, (2) E ⟼ T, (3) T ⟼ x.

What info are needed to resolve the LR(0) conflicts?

Follow sets: symbols after T



Look ahead a symbol in input.

FOLLOW Sets

- SLR parsing utilizes a lookahead string of length 1
  - Use FOLLOW sets – the set of terminals that may immediately follow a terminal or non-terminal in some derivation of a string
  - Used to determine when a reduction should be applied

- To construct FOLLOW sets:
  1. place $ in FOLLOW(S')
  2. for A $\rightarrow \alpha B \beta$ add FIRST(β)-{ε} to FOLLOW(B)
  3. for A $\rightarrow \alpha B$ add FOLLOW(A) to FOLLOW(B)
  4. for A $\rightarrow \alpha B \beta$ if $\varepsilon \in$ FIRST(β) add FOLLOW(A) to FOLLOW(B)

Computing First Sets

- FIRST sets

  For some rhs of a production α, FIRST(α) is the set of tokens (terminals) that appear as the first symbol in some string derived from α

$$x \in \text{FIRST}(\alpha) \Leftrightarrow \exists x \in \Sigma \mid \alpha \Rightarrow^* x\gamma$$

  To compute FIRST(X)

```
if X is a terminal then
   FIRST(X) = {X}
else if X → ε then
   FIRST(X) ∪= {ε}
else if X → Y1Y2 … Yk then
   FIRST(X) ∪= FIRST(Y1)
   ∀i | ε ∈ FIRST(Yj), 1 ≤ j < i, FIRST(X) ∪= FIRST(Yi)
```

Computing FIRST Sets

- Initially, for all nonterminals **A**, set

  $$\text{FIRST}(\mathbf{A}) = \{\ \mathtt{t} \mid \mathbf{A} \rightarrow \mathtt{t}\omega \text{ for some } \omega\ \}$$

- Then, repeat the following until no changes occur: For each nonterminal **A**, for each production $\mathbf{A} \rightarrow \mathbf{B}\omega$, set

  $$\text{FIRST}(\mathbf{A}) = \text{FIRST}(\mathbf{A}) \cup \text{FIRST}(\mathbf{B})$$

- This is known a **fixed-point iteration** or a **transitive closure algorithm**.

Computing FIRST Sets

```
STMT  →  if EXPR then STMT          (1)
      |  while EXPR do STMT         (2)
      |  EXPR ;                     (3)
EXPR  →  TERM -> id                 (4)
      |  zero? TERM                 (5)
      |  not EXPR                   (6)
      |  ++ id                      (7)
      |  -- id                      (8)
TERM  →  id                         (9)
      |  constant                   (10)
```

- Initially, for all nonterminals **A**, set

  FIRST(**A**) = { **t** | **A** → **t**ω for some ω,

  t is a terminal symbol }

- Then, repeat the following until no changes occur: For each nonterminal **A**, for each production **A** → **B**ω, set
- FIRST(**A**) = FIRST(**A**) ∪ FIRST(**B**)

This is known a **fixed-point iteration** or a **transitive closure algorithm**.

Iterative FIRST Computations

```
STMT  →  if EXPR then STMT
      |  while EXPR do STMT
      |  EXPR ;

EXPR  →  TERM -> id
      |  zero? TERM
      |  not EXPR
      |  ++ id
      |  -- id

TERM  →  id
      |  constant
```

Iterative FIRST Computations

Initially, for all nonterminals **A**,

set FIRST(**A**) = { **t** | **A** → **t**ω for some ω }

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | const
```

STMT	EXPR	TERM

Iterative FIRST Computations

Initially, for all nonterminals **A**,

set FIRST(**A**) = { **t** | **A** → **t**ω for some ω }

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | constan
       t
```

STMT	EXPR	TERM
if		
while		

Iterative FIRST Computations

Initially, for all nonterminals **A**,

set FIRST(**A**) = { **t** | **A** → **t**ω for some ω }

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     |
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	
while	not	
	++	
	--	

Iterative FIRST Computations

Initially, for all nonterminals **A**,

set FIRST(**A**) = { **t** | **A** → **t**ω for some ω }

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
	++	
	--	

Iterative FIRST Computations

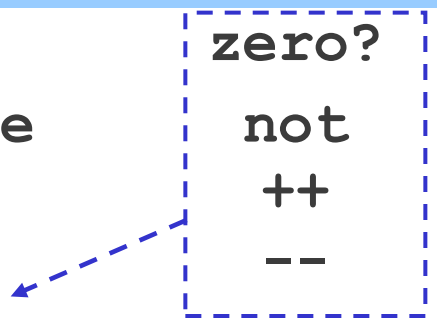
Repeat the following until no changes occur: For each production **A** → **B** ω, set FIRST(**A**) = FIRST(**A**) ∪ FIRST(**B**)

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
	++	
	--	



STMT depends on EXPR

Iterative FIRST Computations

Repeat the following until no changes occur: For each production **A** → **B** ω, set FIRST(**A**) = FIRST(**A**) ∪ FIRST(**B**)

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++		
--		



STMT depends on EXPR

Iterative FIRST Computations

Repeat the following until no changes occur: For each production $A \rightarrow B\,\omega$, set FIRST(A) = FIRST(A) ∪ FIRST(B)

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++		
--		

Iterative FIRST Computations

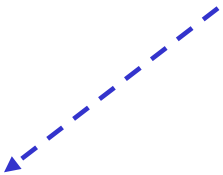
Repeat the following until no changes occur: For each production **A** → **B** ω, set FIRST(**A**) = FIRST(**A**) ∪ FIRST(**B**)

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++		
--		



EXPR depends on TERM

Iterative FIRST Computations

Repeat the following until no changes occur: For each production **A** → **B** ω, set FIRST(**A**) = FIRST(**A**) ∪ FIRST(**B**)

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	

EXPR depends on TERM

Iterative FIRST Computations

Repeat the following until no changes occur: For each production $A \rightarrow B\,\omega$, set FIRST(A) = FIRST(A) ∪ FIRST(B)

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	

Iterative FIRST Computations

Repeat the following until no changes occur: For each production **A** → **B** ω, set FIRST(**A**) = FIRST(**A**) ∪ FIRST(**B**)

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	

STMT depends on EXPR

Iterative FIRST Computations

Repeat the following until no changes occur: For each production **A** → **B** ω, set FIRST(**A**) = FIRST(**A**) ∪ FIRST(**B**)

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

STMT depends on EXPR

Iterative FIRST Computations

```
STMT → if EXPR then STMT
     | while EXPR do STMT
     | EXPR ;

EXPR → TERM -> id
     | zero? TERM
     | not EXPR
     | ++ id
     | -- id

TERM → id
     | consta
       nt
```

STMT	EXPR	TERM
if	zero?	id
while	not	constant
zero?	++	
not	--	
++	id	
--	constant	
id		
constant		

In-class exercise

Show the first sets for non-terminal symbols in the following grammar.

E --> TE'
E' --> +TE' | **ε**
T --> FT'
T' --> *FT' | **ε**
F --> id | (E)

FOLLOW Sets

- SLR parsing utilizes a lookahead string of length 1
  - Use FOLLOW sets – the set of terminals that may immediately follow a terminal or non-terminal in some derivation of a string
  - Used to determine when a reduction should be applied

- To construct FOLLOW sets:
  1. place $ in FOLLOW(S')
  2. for A $\rightarrow \alpha B \beta$ add FIRST(β)-{ε} to FOLLOW(B)
  3. for A $\rightarrow \alpha B$ add FOLLOW(A) to FOLLOW(B)
  4. for A $\rightarrow \alpha B \beta$ if $\varepsilon \in$ FIRST(β) add FOLLOW(A) to FOLLOW(B)

Compute the follow sets for Sign

If **B** → *α* **A** *ω* is a production, set FOLLOW(**A**) = FOLLOW(**A**) ∪ FIRST*(*ω*) - { **ε** }.

Num → **Sign Digits**
Sign → **+ − ε**
Digits → **Digit More**
More → **Digits | ε**
Digit → **0 | 1 | … | 9**

First set

Num		Sign		Digit		Digits		More	
+	−	+	−	0	5	0	5	0	5
0	5	ε		1	6	1	6	1	6
1	6			2	7	2	7	2	7
2	7			3	8	3	8	3	8
3	8			4	9	4	9	4	9
4	9							ε	

Follow(Sign)

Look at the Num → **Sign Digits**,

Follow(Sign) = Follow(Sign) ∪ *First(Digits)*

Follow(Sign) = {0,…,9}

Compute the follow set for More

If **B** → *α* **A** *ω* is a production and **ε** ∈ FIRST*(*ω*), set FOLLOW(**A**) = FOLLOW(**A**) ∪ FOLLOW(**B**).

Num → **Sign Digits**
Sign → **+** | **-** | **ε**
Digits → **Digit More**
More → **Digits** | **ε**
Digit → **0** | **1** | … | **9**

First set

Num		Sign		Digit		Digits		More	
+	-	+	-	0	5	0	5	0	5
0	5		ε	1	6	1	6	1	6
1	6			2	7	2	7	2	7
2	7			3	8	3	8	3	8
3	8			4	9	4	9	4	9
4	9								ε

Follow(More)

Look at the **Digits** → **Digit More**

Look at follow(Digits)

follow(More) = follow(Digits) = follow(Num)

Compute the follow set for Digits

If **B** → *α* **A** *ω* is a production and **ε** ∈ FIRST*(*ω*), set FOLLOW(**A**) = FOLLOW(**A**) ∪ FOLLOW(**B**).

Num **Sign Digits**
Sign **+** **-** **ε**
Digits → **Digit More**
More → **Digits** | **ε**
Digit → **0** | **1** | … | **9**

First set

Num		Sign		Digit		Digits		More	
+	-	+	-	0	5	0	5	0	5
0	5		ε	1	6	1	6	1	6
1	6			2	7	2	7	2	7
2	7			3	8	3	8	3	8
3	8			4	9	4	9	4	9
4	9							ε	

Follow(Digits)

Look at the **Num** → **Sign Digits**

look at follow(Digits)

follow(Digits)=follow{Num}={$}

follow(More) = follow(Digits) = follow(Num) ={$}

Compute the follow set for Digit

If **B** → *a* **A** *ω* is a production and **ε** ∈ FIRST*(*ω*), set FOLLOW(**A**) = FOLLOW(**A**) ∪ FOLLOW(**B**).

Num **Sign Digits**
Sign **+ - ε**
Digits → **Digit More**
More → **Digits | ε**
Digit → **0 | 1 | … | 9**

First set

Num		Sign		Digit		Digits		More	
+	-	+	-	0	5	0	5	0	5
0	5		ε	1	6	1	6	1	6
1	6			2	7	2	7	2	7
2	7			3	8	3	8	3	8
3	8			4	9	4	9	4	9
4	9							ε	

Follow(Digit)

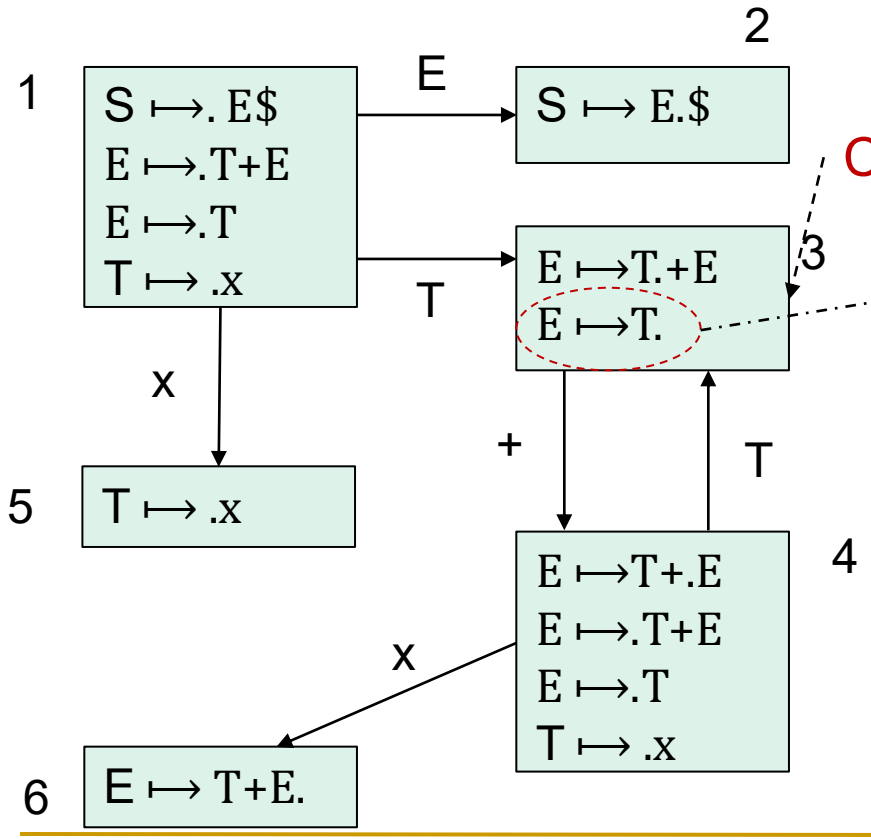
Look at the **Digits → Digit More**,
follow(Digit) ⊇first(More) - {ε}

Since **More → ε**, check follow(Num)
follow(Digit) ⊇follow(Digits)={$}

Follow(Digit)= {0,1,…,9,$}

Resolving LR(0) conflicts using the follow sets

Grammar (0) S ⟼ E \$, (1) E ⟼ T + E,
(2) E ⟼ T, (3) T ⟼ x.



	x	+	\$	E	T
1	s5			g2	g3
2			a		
3	r2	s4,r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		

Conflict

Before resolving conflict

Follow(E) = {\$}

	x	+	\$	E	T
1	s5			g2	g3
2			a		
3		s4	r2		
4	s5			g6	g3
5		r3	r3		
6			r1		

After resolving conflict

In-class Exercise

Show the follow sets for non-terminal symbols in the following grammar.

E --> TE'
E' --> +TE' | **ε**
T --> FT'
T' --> *FT' | **ε**
F --> id | (E)

FIRST(F) = { **id**, **(** } since F → id | (E)
FIRST(T′) = { *, **ε** } since T′ → * F T′ | ε
FIRST(T) = { **id**, **(** } since T → F T′ and FIRST(F) = {id,(}
FIRST(E′) = { **+**, **ε** } since E′ → + T E′ | ε
FIRST(E) = { **id**, **(** } since E → T E′ and FIRST(T) = {id,(}

In-class Exercise

Show the follow sets for non-terminal symbols in the following grammar.

E --> TE'
E' --> +TE' | **ε**
T --> FT'
T' --> *FT' | **ε**
F --> id | (E)

FIRST(F) = { **id**, **(** } since F → id | (E)
FIRST(T′) = { *, **ε** } since T′ → * F T′ | ε
FIRST(T) = { **id**, **(** } since T → F T′ and FIRST(F) = {id,(}
FIRST(E′) = { **+**, **ε** } since E′ → + T E′ | ε
FIRST(E) = { **id**, **(** } since E → T E′ and FIRST(T) = {id,(}

Step 1: FOLLOW(E)
•Since **E** is the start symbol:
FOLLOW(E) = { **$** }
•From **F** → **(E)**: after E comes), so) ∈ FOLLOW(E).
→ FOLLOW(E) = { **$,)** }

Step 2: FOLLOW(E′)
•From **E** → **T E′**: E′ is at the end → FOLLOW(E′) = FOLLOW(E).
→ FOLLOW(E′) = { **$,)** }

Step 3: FOLLOW(T)
•From **E** → **T E′**: after T comes E′ → FIRST(E′) = {+, ε}.
- Add + to FOLLOW(T).
- Since E′ → ε, also add FOLLOW(E) = { $,) }.
  → FOLLOW(T) = { **+, $,)** }

Step 4: FOLLOW(T′)
•From **T** → **F T′**: T′ is at the end → FOLLOW(T′) = FOLLOW(T).
→ FOLLOW(T′) = { **+, $,)** }

Step 5: FOLLOW(F)
•From **T** → **F T′**: after F comes T′ → FIRST(T′) = {*, ε}.
- Add * to FOLLOW(F).
- Since T′ → ε, also add FOLLOW(T) = {+, $,)}.
  → FOLLOW(F) = { ***, +, $,)** }

LR Parsing

- LR(1) parsing (Left-to-right scan, Reverse of the rightmost derivation, 1 symbol of lookahead)
  - table-driven shift-reduce parsing
  - characteristic DFA plus a stack

- SLR (Simple LR) grammar without conflicts in the parsing table.
  - Conflicts can be resolved with first/follow sets.
  - Grammars with conflicts that can not be resolved are ***non-SLR***.
- Automatic parser generators (yacc, bison and lemon generate a subset of LR(1) known as LALR(1))

$$LR(1) \supset LALR(1) \supset SLR$$

- Note:

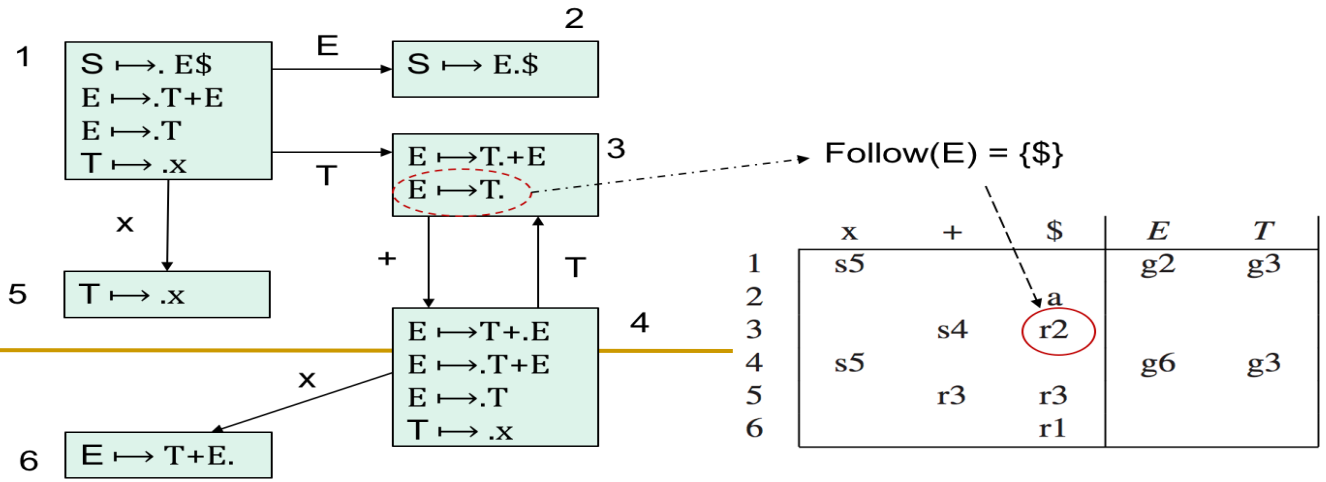
SLR Table Construction

1. Construct the set of LR(0) items, C
2. Construct state s_i from $I_i \in$ C as follows

   // I_i becomes s_i

   1. If $A \rightarrow \alpha \cdot a\beta \in I_i$ and goto(I_i,a) == I_j and $a \in T$ then
      action[i,a] = "shift j"
   2. If $A \rightarrow \alpha \cdot \in I_i$ then
      $\forall a \in FOLLOW(A) \mid A \neq S'$, action[i,a] = "reduce $A \rightarrow \alpha$"
   3. If $S' \rightarrow S \cdot \in I_i$ then
      action[i,$] = "accept"
   4. If $A \rightarrow \alpha \cdot B\beta \in I_i$ and goto(I_i,B) == I_j and $B \in N$ then
      GOTO[i,B] = j

S' : start symbol



Errors in Constructing the Parse Table

- shift/reduce conflict - the input grammar is ambiguous
  - rewrite the grammar to be unambiguous or use precedence

S → if E then S
  | if E then S else S

consider parsing
```
if x then
   if y then print x;
else print y;
```

- reduce/reduce conflict - grammar has two identical rhs with different lhs that appear in same context

Practice Example

Is the following grammar SLR?

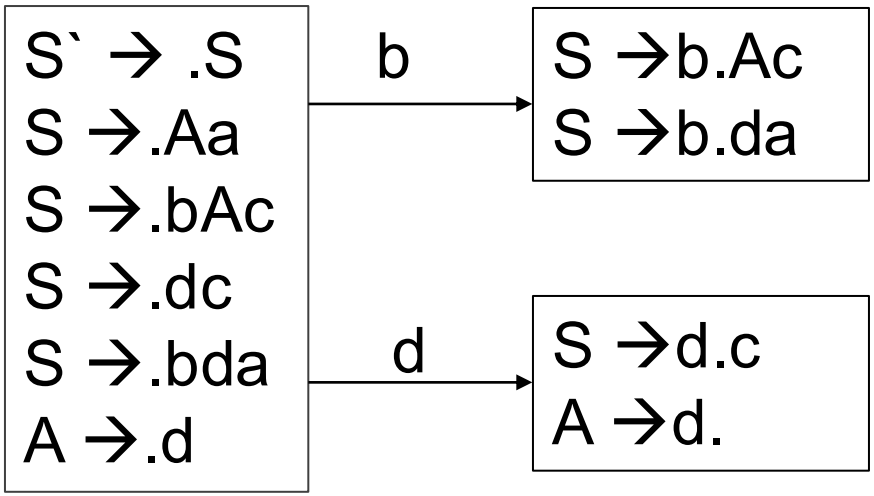
Does there exist shift-reduce or reduce-reduce conflict in its SLR DFA?

$S \rightarrow Aa \mid bAc \mid dc \mid bda$
$A \rightarrow d$

Practice Example

G: S → Aa | bAc | dc | bda A → d

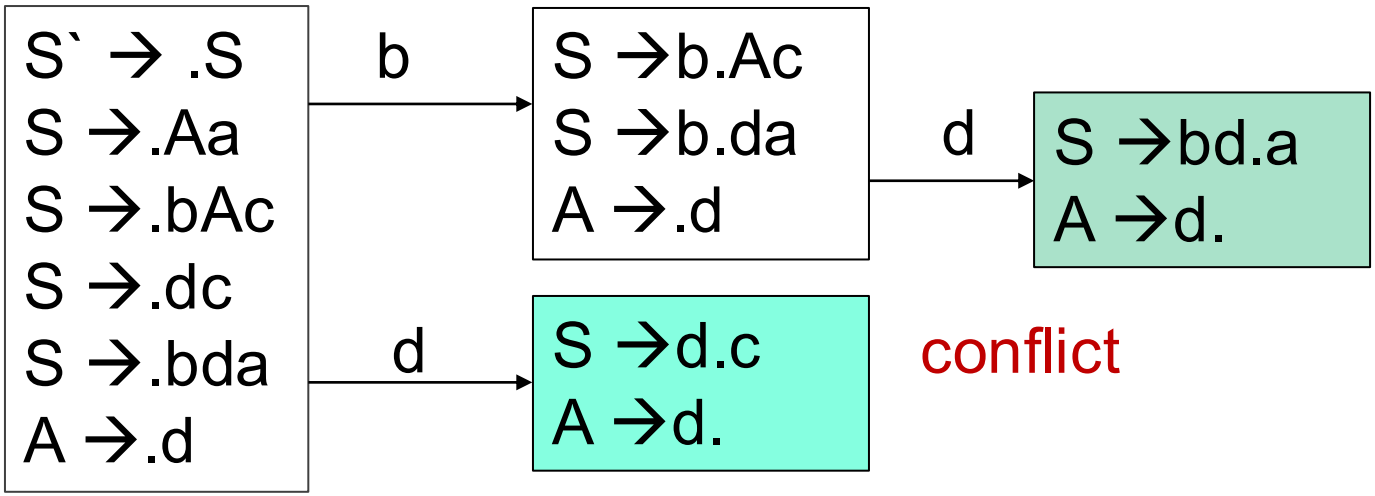
Extend G S` → S



Practice Example

G: S → Aa | bAc | dc | bda A → d

Extend G S` → S

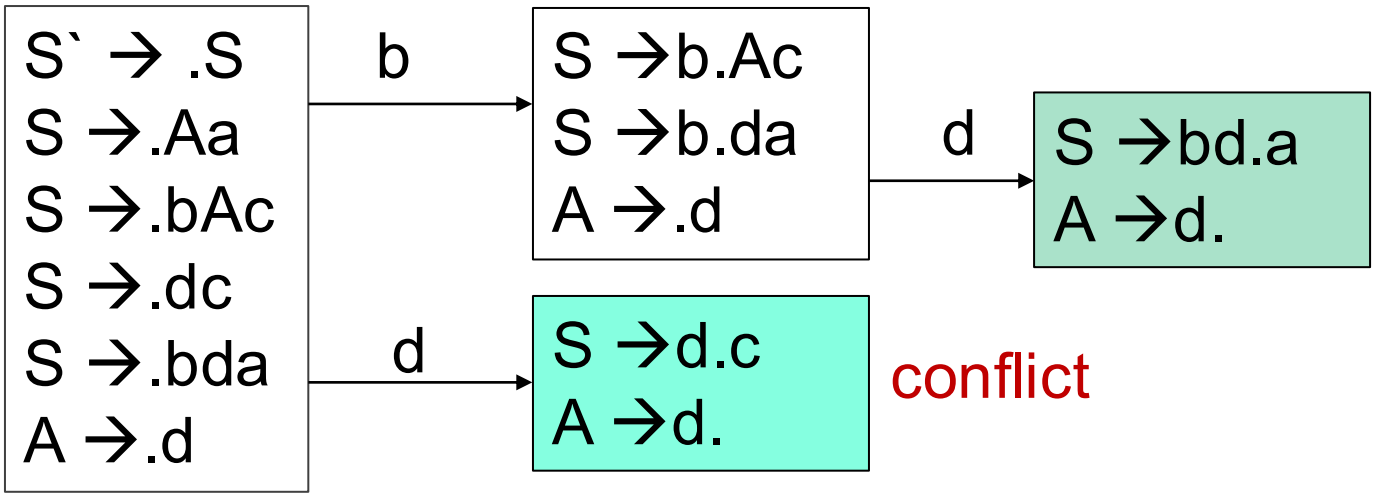


Compute follow(A) = {a, c}

Practice Example

G: S → Aa | bAc | dc | bda A → d

Extend G S` → S



Compute follow(A) = {a, c}
c is after d in item S → d.c, c is in follow(A).
The conflict is not resolved. G is not SLR(1).